SPEEDWAGON LOGISTICS

Derick Fernandes Souta (Linkedin)

Fabiano Almeida Cardoso de Souza (Linkedin)

Jonathan Wesley Ferreira da Silva (Linkedin)

Julio Eduardo Bustamante Mancisidor (Linkedin)

Lucas Daniel da Silva Faria (Linkedin)

Lucas Silva Daniel (Linkedin)

Tiago Moreira Luz Bittencourt (Linkedin)

Professor M2 ou Orientador: Prof. Newton Eizo Yamada

Professor P2: Prof. Marcus Nascimento

Resumo do projeto:

Este projeto tem como objetivo o desenvolvimento de uma solução em Business Intelligence voltada para o controle, análise e acompanhamento da evolução e giro de estoque de materiais na Companhia Paulista de Trens Metropolitanos (CPTM). A proposta inclui a criação de dashboards interativos em Power BI que organizem e integrem as bases de dados do sistema ERP ALVO, como os relatórios de cadastro de materiais, saldos físicos e movimentações históricas, permitindo analisar indicadores de rotatividade, impacto financeiro e riscos operacionais, com foco na prevenção de falta de peças para a manutenção dos trens e na redução de capital parado.

Palavras-Chave: CPTM; Power BI; Logística; Operação; Análise.

Abstract:

This project aims to develop a Business Intelligence solution focused on controlling, analyzing, and tracking the evolution and turnover of material inventory at the Companhia Paulista de Trens Metropolitanos (CPTM). The proposal includes the creation of interactive Power BI dashboards to organize and integrate databases from the ALVO ERP system, such as material registration reports, physical balances, and historical movements. This enables the analysis of turnover indicators, financial impact, and operational risks, focusing on the prevention of part shortages for train maintenance and the reduction of idle capital.

Keywords: CPTM; Power BI; Logistics; Operation; Analysis.

1. Contextualização do projeto

O transporte ferroviário metropolitano desempenha um papel fundamental para a mobilidade e o desenvolvimento socioeconômico no Brasil, atuando como um modal de alta capacidade essencial para o deslocamento populacional em regiões metropolitanas. Nesse cenário, a eficiência da gestão da cadeia de suprimentos e dos estoques de manutenção é um fator crítico de sucesso na logística, pois a falta de peças e materiais pode interromper a operação das linhas, gerar atrasos e elevar os custos operacionais.

Nesse contexto, a Companhia Paulista de Trens Metropolitanos (CPTM) destaca-se como um dos principais operadores de transporte sobre trilhos do país, percorrendo diariamente cerca de 53,5 mil quilômetros em mais de 1,5 mil viagens para atender 12 municípios da Região Metropolitana de São Paulo. Para manter essa infraestrutura em pleno funcionamento, a Gerência de Logística (GOL) administra o fluxo e a armazenagem de materiais de manutenção por meio do sistema ERP ALVO, enfrentando o desafio diário de equilibrar a alta disponibilidade de peças essenciais com a redução de capital financeiro imobilizado em estoque.

Este projeto aborda, por meio do tratamento e da análise de dados de estoque extraídos do ERP ALVO, o desenvolvimento de uma solução de Business Intelligence (BI) composta por dashboards interativos no Power BI. A ferramenta visa otimizar o acompanhamento do giro de estoque, da Curva ABC e dos níveis de cobertura operacional, permitindo a identificação precoce de riscos de falta ou excesso de materiais e fornecendo suporte direto à tomada de decisão estratégica na logística ferroviária.

2. Objetivos do projeto

Este projeto tem como objetivo desenvolver uma ferramenta de Business Intelligence focada na gestão, controle e análise do estoque de materiais de manutenção da CPTM, visando:

i) Integrar e organizar as diferentes bases de dados brutas obtidas do ERP ALVO, como cadastros de materiais, movimentações históricas, saldos e centros de custo;
ii) Analisar os indicadores de desempenho do estoque da CPTM, com foco na velocidade de rotatividade (giro), estoque médio e o valor total do capital imobilizado;
iii) Desenvolver um dashboard interativo no Power BI com visões gerenciais e operacionais, incluindo uma versão otimizada para visualização rápida em celulares e tablets;
iv) Implementar um sistema automático de classificação de importância e alertas visuais de risco para identificar materiais parados há muito tempo, em excesso ou com risco de falta para a manutenção das frotas de trens.

3. Fundamentação das tecnologias utilizadas

Esse tópico apresenta de forma detalhada as tecnologias utilizadas para se cumprir com os objetivos estabelecidos do projeto.

3.1. Tecnologias da Informação

Para o desenvolvimento deste projeto de Business Intelligence voltado à gestão de estoque da CPTM, foram selecionadas duas tecnologias principais da informação: o ecossistema Python (utilizando a biblioteca Pandas no ambiente Google Colab) para o tratamento e preparação dos dados, e o Microsoft Power BI para a modelagem dimensional e construção dos dashboards interativos.

1. Python (Google Colab e Biblioteca Pandas)

- **Motivo de uso:** O Google Colab foi escolhido por ser um ambiente de desenvolvimento em nuvem gratuito, acessível e altamente colaborativo, dispensando instalações locais complexas na infraestrutura da equipe. A biblioteca Pandas foi selecionada por ser a ferramenta padrão para manipulação, limpeza e transformação de grandes volumes de dados tabulares.
- **Aplicação no projeto:** O Python foi utilizado na etapa de extração e tratamento dos dados brutos (ETL) fornecidos pelas planilhas do ERP ALVO (como os relatórios PROD, RES34, RES75, RES82 e Centro de Custo). Com o Pandas, a equipe realizou a conversão de tipos de dados, padronização de datas e valores monetários, remoção de registros nulos e a filtragem lógica de vigência dos centros de custo (separando unidades ativas de inativas).
- **Dificuldades de aplicação:** A principal dificuldade esteve relacionada às inconsistências de formatação presentes nas planilhas de origem do ERP, como células mescladas, variações no formato de datas e o elevado volume de registros nas tabelas de movimentações históricas. Exigiu-se a escrita de scripts cuidadosos para evitar a perda de dados operacionais e garantir que a fusão de tabelas (merge) não gerasse duplicidades nos saldos finais.
- **Contribuição efetiva:** A automação do tratamento via Pandas garantiu a entrega de tabelas perfeitamente limpas, padronizadas e validadas para a etapa seguinte, eliminando divergências matemáticas antes da carga no software de visualização.

2. Microsoft Power BI

- **Motivo de uso:** O Power BI é a ferramenta de Business Intelligence de referência no mercado, reconhecida por sua alta capacidade de processamento analítico, ampla biblioteca visual, linguagem de cálculo dinâmico (DAX) e facilidade na construção de layouts responsivos para computadores e dispositivos móveis.
- **Aplicação no projeto:** No Power BI, foi construída a arquitetura de modelagem dimensional no formato Esquema Estrela (Star Schema), conectando as tabelas de dimensões (dMaterial, dCentroCusto, dCalendario) às tabelas fato (fEstoqueAtual e fMovimentacoes). Foram desenvolvidas medidas em linguagem DAX para calcular indicadores de desempenho como Valor Total Imobilizado, Giro de Estoque, Tempo de Cobertura (Meses) e a classificação automática da Curva ABC. Além disso, foram estruturados visuais interativos com filtros por almoxarifado e

formatação condicional para alertas de risco de ruptura e materiais parados, além de uma versão de layout otimizada para celulares e tablets.

- **Dificuldades de aplicação:** As maiores dificuldades envolveram a escrita de fórmulas DAX avançadas que mantivessem a performance do painel rápida (tempo de resposta inferior a 2 segundos) ao calcular a Curva ABC dinâmica e a reconstrução do saldo acumulado histórico mês a mês, além de ajustar os relacionamentos para evitar contagens duplicadas entre almoxarifados distintos.
- **Contribuição efetiva:** O Power BI transformou dados brutos e complexos em painéis gerenciais e operacionais extremamente intuitivos. A tecnologia viabilizou a entrega de uma ferramenta visual capaz de apoiar os gestores da CPTM na identificação rápida de gargalos, prevenção de falta de peças de manutenção de trens e otimização do capital de giro.

3. Aplicação Web em Nuvem (Base44 - CPTM Insight Flow App)

- **Motivo de uso:** A plataforma Base44 foi selecionada como uma solução moderna de desenvolvimento web no-code/AI-driven, permitindo criar e disponibilizar uma aplicação nativa em nuvem automatizada para centralizar os relatórios e dados de estoque da CPTM sem dependência exclusiva do Power BI Desktop.
- **Aplicação no projeto:** A ferramenta foi utilizada para construir a plataforma CPTM Insight Flow App, um ambiente web onde as bases tratadas do ERP ALVO são centralizadas e gerenciadas. A aplicação oferece uma interface intuitiva para navegação nos relatórios de estoque, controle de permissões de acesso por perfil de usuário e visualização automatizada dos indicadores logísticos em qualquer dispositivo conectado à internet.
- **Dificuldades de aplicação:** As principais dificuldades envolveram a configuração da arquitetura de dados da plataforma e a estruturação dos fluxos de permissão de acesso, garantindo a integridade dos arquivos e a sincronização correta das tabelas sem comprometer a usabilidade da interface.
- **Contribuição efetiva:** A tecnologia entregou uma solução web moderna, acessível e automatizada na nuvem, ampliando a autonomia dos gestores da CPTM ao permitir a consulta prática e centralizada das informações de estoque por meio do navegador.

4. Coleta e descrição dos dados utilizados

4.1. Tratamento e Automação do Cadastro de Produtos (PROD) com Python

O primeiro ciclo de automação e Engenharia de Dados concentrou-se no tratamento da base PROD (cadastro oficial de materiais e produtos da CPTM), que continha dados estruturais, fiscais e administrativos essenciais, mas apresentava alto volume de colunas irrelevantes para a análise logística e presença de valores ausentes. O tratamento foi dividido em duas fases principais dentro do script Python no Google Colab:

Fase 1: Conversão de Formato e Filtragem Inicial de Estrutura

1. **Conexão ao Ambiente e Definição de Diretório:** O script inicia com a montagem do Google Drive (drive.mount) e a definição do caminho para a pasta de arquivos brutos do ERP ALVO

2. **Otimização de Desempenho via Conversão de Formato:** Para contornar a lentidão na leitura repetida de arquivos Excel pesados (.xlsx), o Pandas carrega a planilha original e a exporta para o formato .csv com codificação utf-8-sig e separador de ponto e vírgula ; . Essa conversão reduz o tempo de processamento e estabiliza a leitura dos dados.
3. **Leitura e Inspeção de Tipos:** A base em CSV é recarregada utilizando o parâmetro low_memory=False para tratar colunas com tipos de dados mistos. A exibição de colunas é configurada sem limite (max_columns = None) e o comando .head(1) é executado para inspecionar os nomes dos campos disponíveis.
4. **Remoção de Colunas Irrelevantes (Drop de Atributos):** Uma lista abrangente de colunas desnecessárias para a gestão de estoque — como atributos de espaço físico, aplicações externas, faixas de horário, iluminação, dados tributários de ICMS/ABNT e parâmetros de fábrica — foi removida com o comando .drop(columns=colunas_apagar).

Fase 2: Tratamento de Nulos, Padronização do Texto e Exportação Final 5. Higienização de Valores Ausentes

Todos os registros nulos ou vazios (NaN) foram preenchidos com o padrão 'Sem Informação' por meio do método .fillna(). Em seguida, aplicou-se uma função lambda com .map() para substituir strings vazias ou preenchidas com marcadores como 'N' pelo termo 'Sem Informação', garantindo a uniformidade dos dados para as consultas no Power BI e na aplicação web. 6. Validação Lógica de Nomes Alternativos: O código executa uma contagem para avaliar a integridade dos campos de descrição secundária ('Nome Alternativo 2' e 'Nome Alternativo 3'), verificando se havia dados nessas colunas quando o 'Nome Alternativo 1' estava ausente. 7. Eliminação de Redundâncias e Geração do Arquivo Limpo: Diante da baixa relevância dessas descrições alternativas, as colunas adicionais foram descartadas. Por fim, a base higienizada e reestruturada foi salva como arquivo CSV consolidado (Arquivo1_Final_Teste.csv), pronta para integração com os demais relatórios do ERP.

Figura 1 - Montagem do Google Drive e Conversão da Planilha Excel Bruta (PROD) para CSV

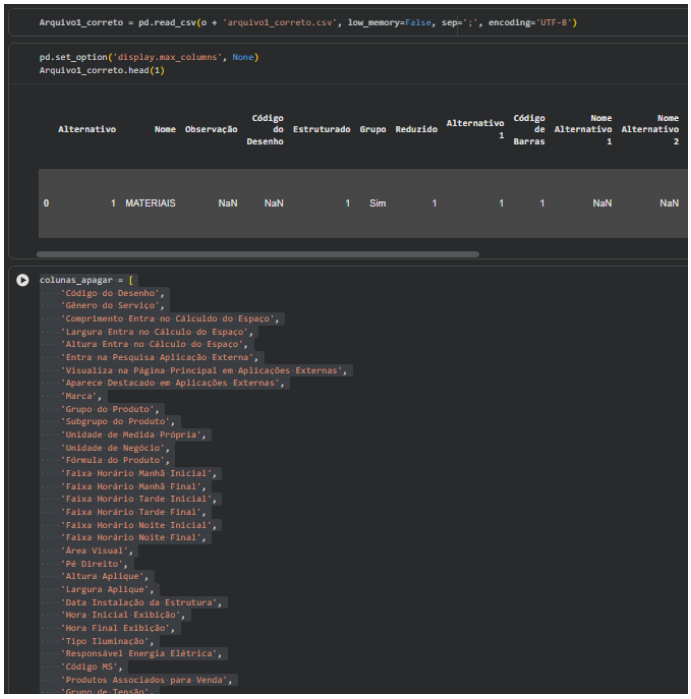
```
import pandas as pd
from google.colab import drive
drive.mount('/content/drive')
o = '/content/drive/My Drive/API/Arquivos brutos/'
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
Arquivo1 = pd.read_excel(o + '1. PROD__20260824.xlsx')
```

```
Arquivo1.to_csv(o + 'arquivo1_correto.csv', index=False, sep=';', encoding='utf-8-sig')
```

Figura 2 - Leitura Otimizada do CSV e Exclusão da Primeira Carga de Colunas Irrelevantes



```
Arquivo1_correto = pd.read_csv(o + 'arquivo1_correto.csv', low_memory=False, sep=';', encoding='UTF-8')

pd.set_option('display.max_columns', None)
Arquivo1_correto.head(1)
```

	Alternativo	Nome	Observação	Código do Desenho	Estruturado	Grupo	Reduzido	Alternativo 1	Código de Barras	Nome Alternativo 1	Nome Alternativo 2
0	1	MATERIAIS	NaN	NaN	1	Sim	1	1	1	NaN	NaN

```
colunas_apagar = [
    'Código do Desenho',
    'Gênero do Serviço',
    'Comprimento Entra no Cálculdo do Espaço',
    'Largura Entra no Cálculo do Espaço',
    'Altura Entra no Cálculo do Espaço',
    'Entra na Pesquisa Aplicação Externa',
    'Visualiza na Página Principal em Aplicações Externas',
    'Aparece Destacado em Aplicações Externas',
    'Marca',
    'Grupo do Produto',
    'Subgrupo do Produto',
    'Unidade de Medida Própria',
    'Unidade de Negócio',
    'Fórmula do Produto',
    'Faixa Horário Manhã Inicial',
    'Faixa Horário Manhã Final',
    'Faixa Horário Tarde Inicial',
    'Faixa Horário Tarde Final',
    'Faixa Horário Noite Inicial',
    'Faixa Horário Noite Final',
    'Área Visual',
    'Pé Direito',
    'Altura Aplique',
    'Largura Aplique',
    'Data Instalação da Estrutura',
    'Hora Inicial Exibição',
    'Hora Final Exibição',
    'Tipo Iluminação',
    'Responsável Energia Elétrica',
    'Código MS',
    'Produtos Associados para Venda',
    'Grupo de Tensão',
```

Figura 3 - Tratamento de Valores Ausentes, Padronização para 'Sem Informação' e Filtro de Nomes Alternativos



```
Arquivo_correto = Arquivo_correto.fillna('Sem Informação')
Arquivo_correto = Arquivo_correto.map(
    lambda x: 'Sem Informação'
    if isinstance(x, str) and (x.strip() == '' or x.strip().upper() == 'N')
    else x
)

pd.set_option('display.max_columns', None)
Arquivo_correto.head(5)

sem_info = 'Sem Informação'
alt2_sem_alt1 = Arquivo_correto[
    (Arquivo_correto['Nome Alternativo 2'] != sem_info) &
    (Arquivo_correto['Nome Alternativo 1'] == sem_info)
]
alt3_sem_alt1 = Arquivo_correto[
    (Arquivo_correto['Nome Alternativo 3'] != sem_info) &
    (Arquivo_correto['Nome Alternativo 1'] == sem_info)
]
alt3_sem_alt2 = Arquivo_correto[
    (Arquivo_correto['Nome Alternativo 3'] != sem_info) &
    (Arquivo_correto['Nome Alternativo 2'] == sem_info)
]
print("Linhas com Alternativo 2, mas sem Alternativo 1:", len(alt2_sem_alt1))
print("Linhas com Alternativo 3, mas sem Alternativo 1:", len(alt3_sem_alt1))
print("Linhas com Alternativo 3, mas sem Alternativo 2:", len(alt3_sem_alt2))

colunas = ['Nome Alternativo 2', 'Nome Alternativo 3']

Arquivo_correto= Arquivo_correto.drop(columns=colunas)

print("Colunas apagadas com sucesso!")
```

Figura 4 - Remoção de Colunas Redundantes e Exportação da Base Final Higienizada



```
pd.set_option('display.max_columns', None)
Arquivo_correto.head(4)

Arquivo_correto.to_csv(o + 'Arquivo1_Final_Teste.csv', index=False, sep=';', encoding='utf-8-sig')

print("Arquivo salvo como arquivo1_correto.csv!")
```

4.1.1. Padronização e Unificação do Relatório de Planejamento e Tipo de Consumo (RES43: Essencial, Estratégico e Normal)

O segundo fluxo de automação em Python tratou os dados do relatório RES43 (relatório de planejamento de consumo de materiais do ERP ALVO). Na origem, as informações eram divididas em três arquivos distintos segundo a criticidade operacional dos itens: Essencial, Estratégico e Normal. Para viabilizar uma análise consolidada no Power BI e no site, desenvolveu-se um script estruturado em três etapas:

Fase 1: Padronização Individual das Três Bases

1. **Carregamento das Bases e Leitura com Separador Personalizado:** Cada um dos três arquivos (arquivo2_corretoV2.csv, arquivo3_corretoV2.csv e arquivo4_corretoV2.csv) foi lido individualmente com separador de ponto e vírgula ; e codificação utf-8.
2. **Harmonização de Formatos Decimais:** As colunas numéricas de consumo e planejamento — como Consumo Médio Mensal e PCA Mensal (fatia) — continham inconsistências de pontuação decimal (uso misto de pontos e vírgulas). Aplicou-se tratamento com .astype(str).str.replace() e pd.to_numeric(errors='coerce') para converter os valores em números decimais válidos com arredondamento em 2 casas (.round(2)).
3. **Tratamento da Valoração Financeira:** Nas bases Estratégica e Normal, a coluna Previsão $ foi convertida para tipo numérico real, garantindo a correta interpretação do impacto orçamentário previsto.
4. **Exportação de Padrões Intermediários:** Após o ajuste de formatos, cada base foi temporariamente salva como arquivo padrão (arquivo2_padrão.csv, arquivo3_padrão.csv e arquivo4_padrão.csv).

Fase 2: Unificação, Criação de Tag Identificadora e Higienização de Texto 5. Rotulagem de Origem (Flag de Criticidade)

Para preservar a rastreabilidade do tipo de consumo de cada material após a fusão, criou-se uma coluna identificadora chamada 'Arquivo' em cada tabela, atribuindo os rótulos 'Essencial', 'Estratégico' e 'Normal'. 6. Concatenação das Bases (pd.concat): As três tabelas padronizadas foram unificadas em um único DataFrame consolidado (Arquivo_final) por meio do comando pd.concat([Arquivo2, Arquivo3, Arquivo4], ignore_index=True). 7. Higienização de Strings e Nulos: Aplicou-se uma função lambda para remover espaços extras dentro das cadeias de texto (' '.join(x.split())). Em seguida, células vazias, hífens isolados ('-') ou marcadores em branco foram substituídos pelo padrão 'Sem Informação' via .fillna() e expressões regulares (regex=True).

Fase 3: Ajuste Final dos Indicadores Financeiros de Compra e Exportação 8. Formatação de Quantidades e Previsão de Compra

As colunas Quantidade Necessária p/ Compra e Previsão $ do dataset unificado passaram por um último ajuste de conversão numérica com tratamento de vírgulas e arredondamento padrão para 2 casas decimais. 9. Geração do Dataset Consolidado RES43: A base final unificada foi exportada em formato CSV (Arquivo2_Consolidado.csv) com codificação utf-8-sig, pronta para alimentar os indicadores de cobertura e necessidade de reposição no Power BI e no site CPTM Insight Flow.

Figura 5 - Padronização Decimal da Base RES43 Essencial



```
import pandas as pd
from google.colab import drive
drive.mount('/content/drive')
o = '/content/drive/My Drive/API/'
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
arquivo2 = pd.read_csv(o + 'arquivo2_corretoV2.csv', low_memory=False, sep=';', encoding='UTF-8')
```

```
pd.set_option('display.max_columns', None)
arquivo2.head(1)
```

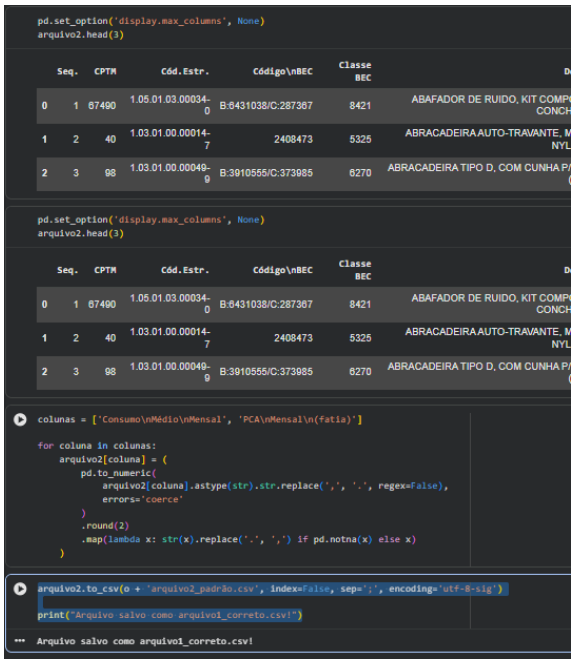
	Seq.	CPTM	Cód.Estr.	Código\nBEC	Classe BEC	Descrição	
0	1	67490	1.05.01.03.00034-0	B:6431038/C:287367	8421	ABAFADOR DE RUIDO, KIT COMPOSTO DE CONCHAS, HA...	2.ET.DM.08 - EQUIPAMEN

```
colunas = ['Consumo\nMédio\nMensal', 'PCA\nMensal\n(fatia)']
for coluna in colunas:arquivo2[coluna] = (arquivo2[coluna].astype(str).str.replace('.', ',', regex=False))
```

```
colunas = ['Consumo\nMédio\nMensal', 'PCA\nMensal\n(fatia)']

for coluna in colunas:
    arquivo2[coluna] = pd.to_numeric(
        arquivo2[coluna].astype(str).str.replace(',', '.', regex=False),
        errors='coerce'
    ).round(2)
```

Figura 6 - Padronização Decimal da Base RES43 Essencial Parte 2



```
pd.set_option('display.max_columns', None)
arquivo2.head(3)
```

	Seq.	CPTM	Cód.Estr.	Código\nBEC	Classe BEC	D
0	1	67490	1.05.01.03.00034-0	B:6431038/C:287367	8421	ABAFADOR DE RUIDO, KIT COMP CONCH
1	2	40	1.03.01.00.00014-7	2408473	5325	ABRACADEIRA AUTO-TRAVANTE, M NYL
2	3	98	1.03.01.00.00049-9	B:3910555/C:373985	6270	ABRACADEIRA TIPO D, COM CUNHA P/


```
pd.set_option('display.max_columns', None)
arquivo2.head(3)
```

	Seq.	CPTM	Cód.Estr.	Código\nBEC	Classe BEC	D
0	1	67490	1.05.01.03.00034-0	B:6431038/C:287367	8421	ABAFADOR DE RUIDO, KIT COMP CONCH
1	2	40	1.03.01.00.00014-7	2408473	5325	ABRACADEIRA AUTO-TRAVANTE, M NYL
2	3	98	1.03.01.00.00049-9	B:3910555/C:373985	6270	ABRACADEIRA TIPO D, COM CUNHA P/


```
colunas = ['Consumo\nMédio\nMensal', 'PCA\nMensal\n(fatia)']

for coluna in colunas:
    arquivo2[coluna] = (
        pd.to_numeric(
            arquivo2[coluna].astype(str).str.replace(',', '.', regex=False),
            errors='coerce'
        )
        .round(2)
        .map(lambda x: str(x).replace('.', ',') if pd.notna(x) else x)
    )
```

```
arquivo2.to_csv(o + 'arquivo2_padrão.csv', index=False, sep=';', encoding='utf-8-sig')

print("Arquivo salvo como arquivo1_correto.csv!")
```

```
Arquivo salvo como arquivo1_correto.csv!
```

Figura 7 - Padronização Decimal e Ajuste da 'Previsão $' da Base RES43 Estratégico

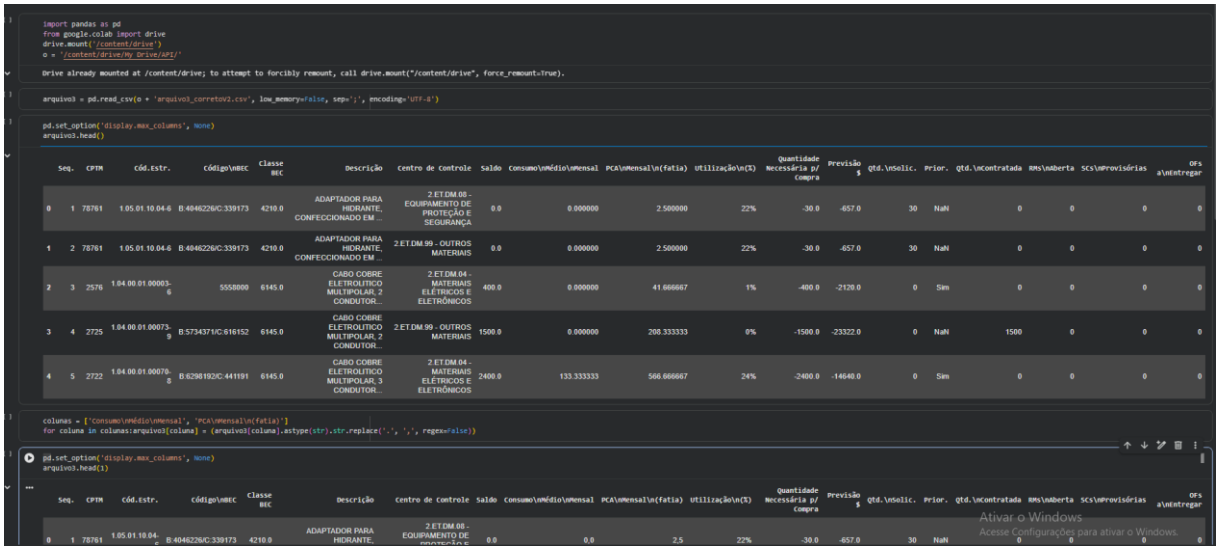


Figura 8 - Padronização Decimal e Ajuste da 'Previsão $' da Base RES43 Estratégico Parte 2

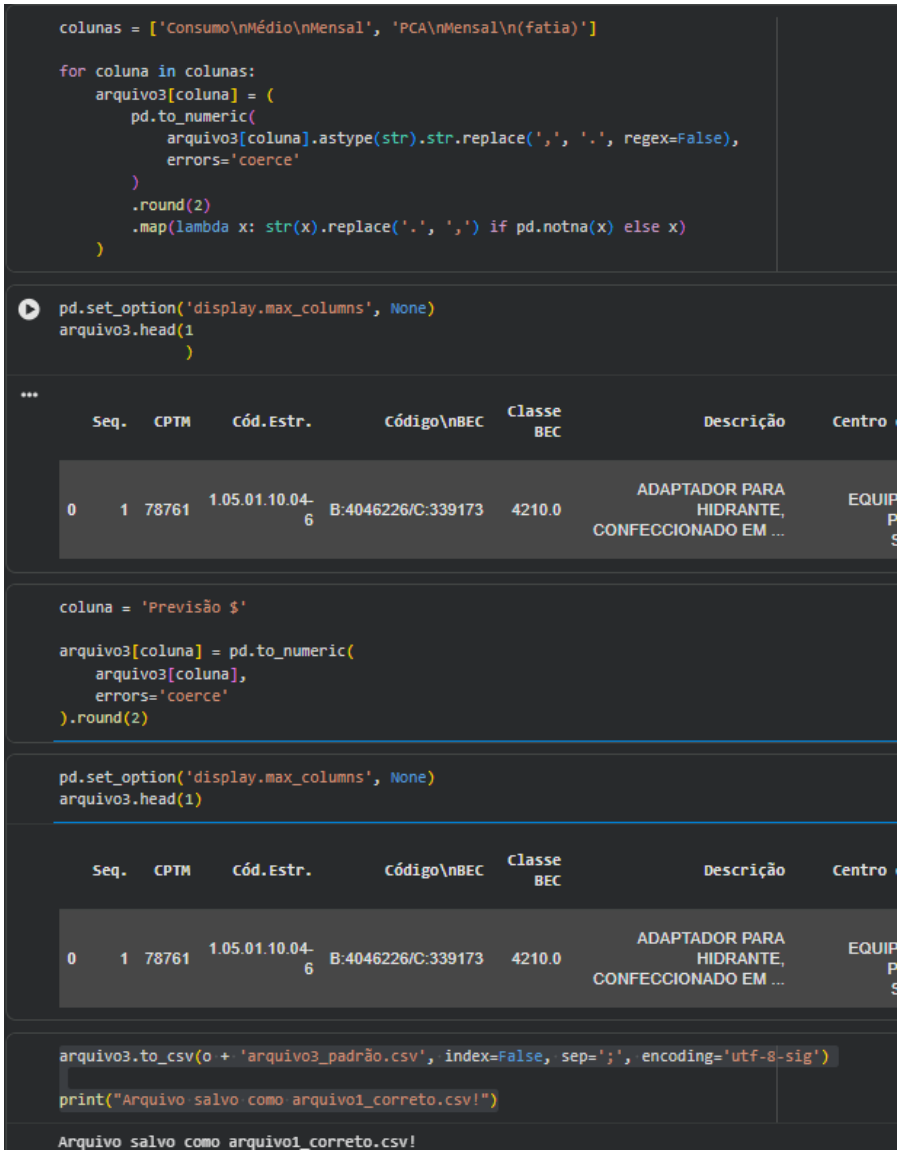
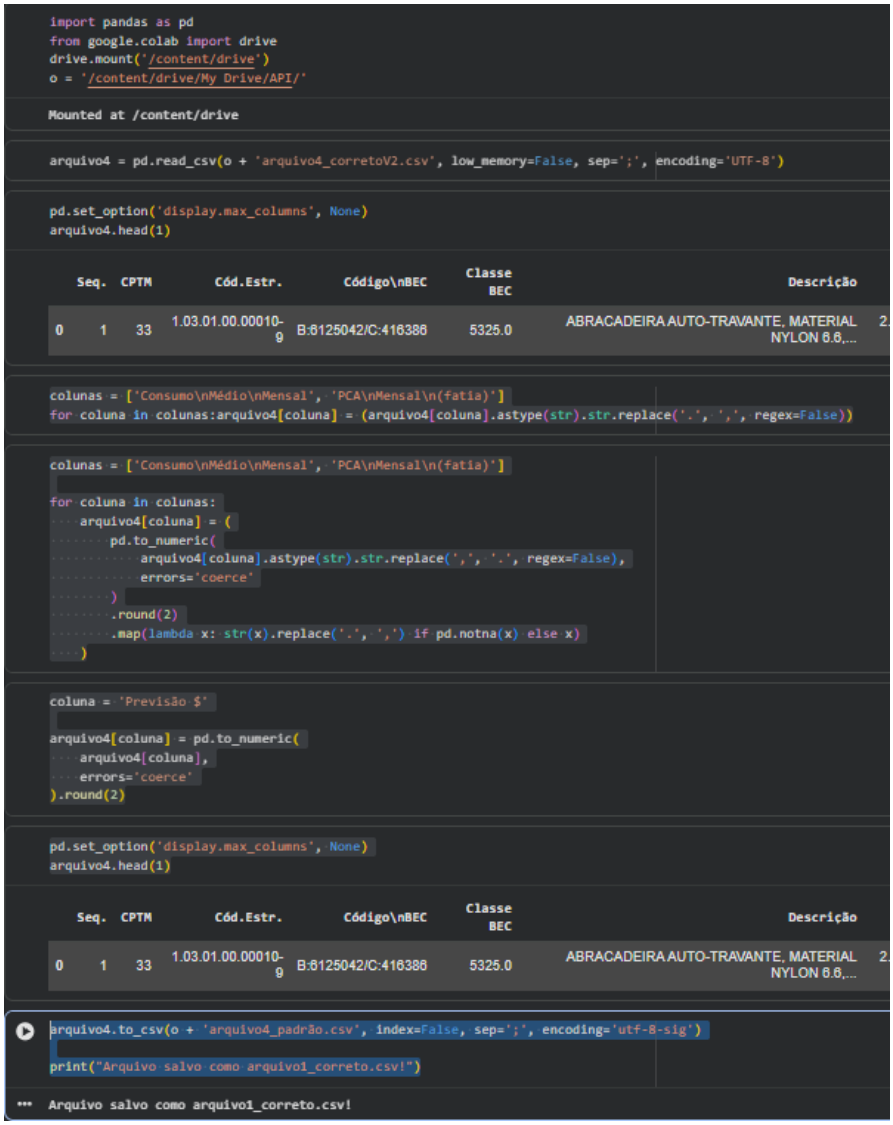


Figura 9 - Padronização Decimal e Ajuste Financeiro da Base RES43 Normal



```
import pandas as pd
from google.colab import drive
drive.mount('/content/drive')
o = '/content/drive/My Drive/API/'
```

```
Mounted at /content/drive
```

```
arquivo4 = pd.read_csv(o + 'arquivo4_corretoV2.csv', low_memory=False, sep=';', encoding='UTF-8')
```

```
pd.set_option('display.max_columns', None)
arquivo4.head(1)
```

	Seq.	CPTM	Cód.Estr.	Código\nBEC	Classe BEC	Descrição	
0	1	33	1.03.01.00.00010-9	B:8125042/C:416388	5325.0	ABRACADEIRA AUTO-TRAVANTE, MATERIAL NYLON 6.6,...	2.

```
colunas = ['Consumo\nMédio\nMensal', 'PCA\nMensal\n(fatia)']
for coluna in colunas:arquivo4[coluna] = (arquivo4[coluna].astype(str).str.replace('.', ',', regex=False))
```

```
colunas = ['Consumo\nMédio\nMensal', 'PCA\nMensal\n(fatia)']

for coluna in colunas:
    arquivo4[coluna] = (
        pd.to_numeric(
            arquivo4[coluna].astype(str).str.replace(',', '.', regex=False),
            errors='coerce'
        )
        .round(2)
        .map(lambda x: str(x).replace('.', ',') if pd.notna(x) else x)
    )
```

```
coluna = 'Previsão $'

arquivo4[coluna] = pd.to_numeric(
    arquivo4[coluna],
    errors='coerce'
).round(2)
```

```
pd.set_option('display.max_columns', None)
arquivo4.head(1)
```

	Seq.	CPTM	Cód.Estr.	Código\nBEC	Classe BEC	Descrição	
0	1	33	1.03.01.00.00010-9	B:8125042/C:416388	5325.0	ABRACADEIRA AUTO-TRAVANTE, MATERIAL NYLON 6.6,...	2.

```
arquivo4.to_csv(o + 'arquivo4_padrão.csv', index=False, sep=';', encoding='utf-8-sig')

print("Arquivo salvo como arquivo1_correto.csv!")
```

```
Arquivo salvo como arquivo1_correto.csv!
```

Figura 10 - Atribuição de Tag, Concatenação (pd.concat) e Exportação Unificada do RES43

```
import pandas as pd
from google.colab import drive
drive.mount('/content/drive')
o = '/content/drive/My Drive/TESTE/'
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content
```

```
Arquivo2 = pd.read_csv(o + 'arquivo2_padrão.csv', low_memory=False, sep=';', encoding='UTF-8')
Arquivo3 = pd.read_csv(o + 'arquivo3_padrão.csv', low_memory=False, sep=';', encoding='UTF-8')
Arquivo4 = pd.read_csv(o + 'arquivo4_padrão.csv', low_memory=False, sep=';', encoding='UTF-8')
```

```
pd.set_option('display.max_columns', None)
Arquivo2.head(1)
```

	Seq.	CPTM	Cód.Estr.	Código\nBEC	Classe BEC	D
0	1	67490	1.05.01.03.00034-0	B:6431038/C:287367	8421	ABAFADOR DE RUIDO, KIT COMP CONCH

```
pd.set_option('display.max_columns', None)
Arquivo3.head(1)
```

	Seq.	CPTM	Cód.Estr.	Código\nBEC	Classe BEC	Desc
0	1	78761	1.05.01.10.04-6	B:4046226/C:339173	4210.0	ADAPTADOR PARA HIDR CONFECCIONADO

```
pd.set_option('display.max_columns', None)
Arquivo4.head(1)
```

	Seq.	CPTM	Cód.Estr.	Código\nBEC	Classe BEC	D
0	1	33	1.03.01.00.00010-9	B:6125042/C:416386	5325.0	ABRACADEIRA AUTO-TRAVANTE, M NYL

```
Arquivo2['Arquivo'] = 'Essencial'
Arquivo3['Arquivo'] = 'Estratégico'
Arquivo4['Arquivo'] = 'Normal'
```

```
pd.set_option('display.max_columns', None)
Arquivo4.head(3)
```

	Seq.	CPTM	Cód.Estr.	Código\nBEC	Classe BEC	Des
0	1	33	1.03.01.00.00010-9	B:6125042/C:416386	5325.0	ABRACADEIRA AUTO-TRAVANTE, MA NYLO
1	2	36	1.03.01.00.00013-0	B:4669916/C:261639	5325.0	ABRACADEIRA AUTO-TRAVANTE, MA NYLO
2	3	91767	1.04.05.08.00159-6	B:6140467/C:377550	5690.0	ACIONADOR DE FECHADURA EL UNIVERSAL

```
Arquivo_final = pd.concat([Arquivo2, Arquivo3, Arquivo4], ignore_index=True)
```

Figura 11 - Atribuição de Tag, Concatenação (pd.concat) e Exportação Unificada do RES43 Parte 2



```
pd.set_option('display.max_columns', None)
Arquivo_final.tail(1)
```

```
Mostrar saída oculta
```

```
Arquivo_final = Arquivo_final.map(
    lambda x: ' '.join(x.split()) if isinstance(x, str) else x
)

Arquivo_final = Arquivo_final.fillna('Sem Informação')

Arquivo_final = Arquivo_final.replace(r'^\s*$', 'Sem Informação', regex=True)

Arquivo_final = Arquivo_final.replace(r'^\s*-\s*$', 'Sem Informação', regex=True)
```

```
pd.set_option('display.max_columns', None)
Arquivo_final.head(20)
```

```
Mostrar saída oculta
```

```
colunas = ['Quantidade Necessária p/ Compra', 'Previsão $']
for coluna in colunas:Arquivo_final[coluna] = (Arquivo_final[coluna].astype(str).str.replace('.', ',', regex=False))
```

```
colunas = ['Quantidade Necessária p/ Compra', 'Previsão $']

for coluna in colunas:
    Arquivo_final[coluna] = pd.to_numeric(
        Arquivo_final[coluna].astype(str).str.replace(',', '.', regex=False),
        errors='coerce'
    ).round(2)
```

```
colunas = ['Quantidade Necessária p/ Compra', 'Previsão $']

for coluna in colunas:
    Arquivo_final[coluna] = (
        pd.to_numeric(
            Arquivo_final[coluna].astype(str).str.replace(',', '.', regex=False),
            errors='coerce'
        )
        .round(2)
        .map(lambda x: str(x).replace('.', ',') if pd.notna(x) else x)
    )
```

```
Arquivo_final.to_csv(o + 'ARTDUWHD.csv', index=False, sep=';', encoding='utf-8-sig')

print("Arquivo salvo como arquivo1_correto.csv!")
```

```
Arquivo salvo como arquivo1_correto.csv!
```

4.1.3. Tratamento e Automação do Cadastro de Produtos (PROD) com Python

O terceiro bloco do processo de Engenharia de Dados tratou a base de Centros de Custo da CPTM (Arquivo 3), responsável por mapear a estrutura organizacional e identificar as unidades responsáveis pelo consumo de materiais. Como essa base continha registros históricos acumulados ao longo de anos e misturava unidades ativas e inativas, o tratamento foi estruturado em dois scripts no Google Colab:

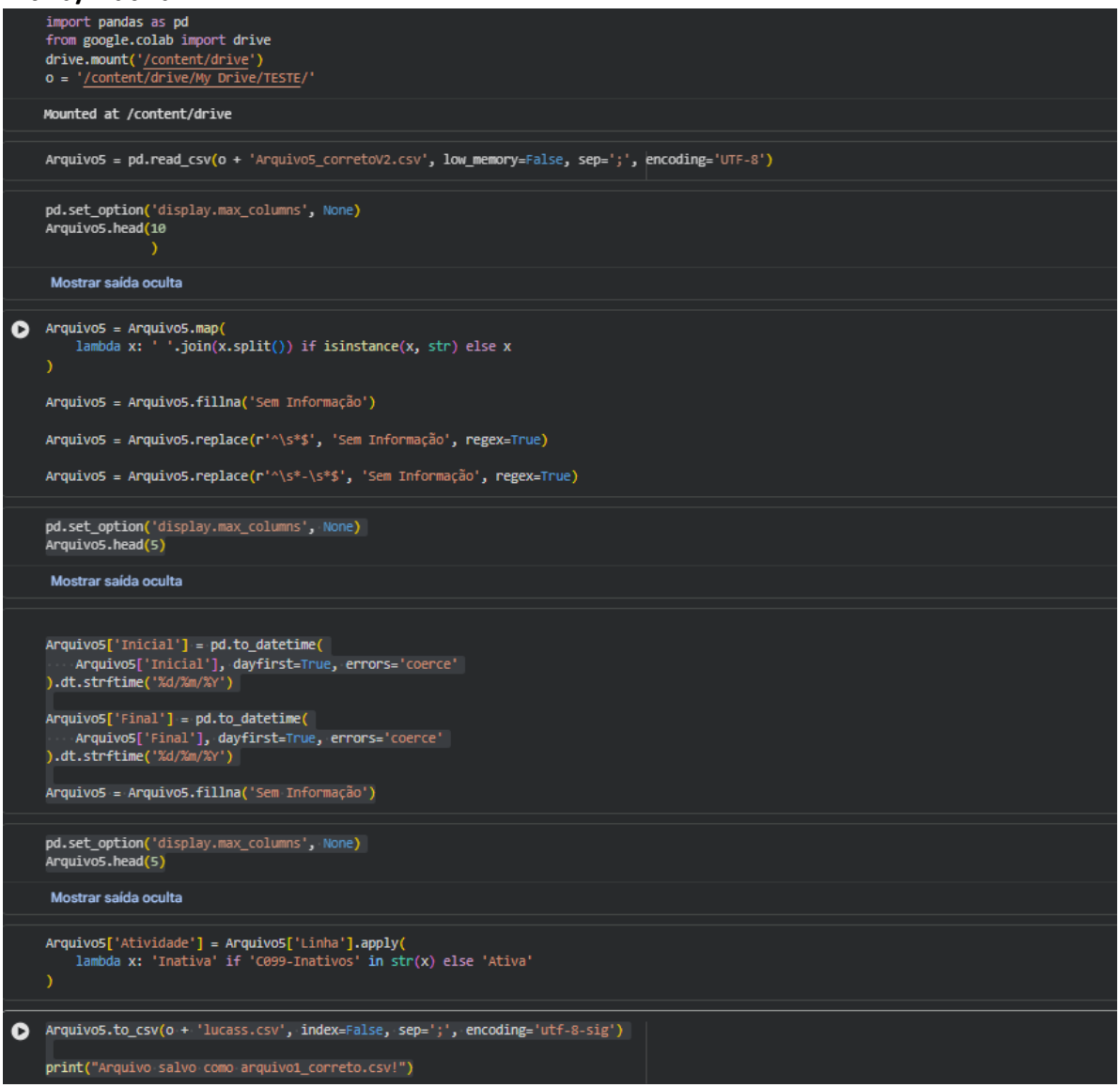
1. Higienização, Padronização de Vigência e Classificação de Atividade

- **Importação e Limpeza Inicial de Texto:** A planilha Arquivo5_corretoV2.csv foi carregada no ambiente. Aplicou-se uma função lambda para eliminar espaços duplicados e quebras de linha nas células de texto. Campos nulos, vazios ou preenchidos com hífens foram padronizados para o termo 'Sem Informação'.
- **Padronização das Datas de Vigência:** As colunas de vigência Inicial e Final foram convertidas para o formato de data brasileiro ('%d/%m/%Y') utilizando pd.to_datetime(..., dayfirst=True).
- **Regra de Negócio para Status de Atividade:** Para permitir a rápida filtragem de unidades operacionais no Power BI e no site, criou-se a coluna Atividade. O código aplica uma regra condicional verificando a coluna Linha: registros associados a 'C099-Inativos' foram classificados como 'Inativa', enquanto os demais foram definidos como 'Ativa'.

2. Filtragem Temporal e Exclusão do Histórico Antigo

- **Conversão Temporária para Datetime:** Para viabilizar operações lógicas com datas, as colunas Inicial e Final foram convertidas temporariamente para tipos de dados datetime (Inicial_dt e Final_dt).
- **Regra de Exclusão por Intervalo de Anos:** A equipe identificou a necessidade de desconsiderar dados operacionais obsoletos do histórico antigo. Foi criada uma lista de exclusão contendo o intervalo de anos de 2002 a 2021 (anos_para_excluir).
- **Filtragem Lógica e Exportação da Base Limpa:** O dataset foi filtrado mantendo apenas as linhas cujas datas de início e término não pertencessem ao intervalo de anos excluídos. Por fim, as colunas temporárias foram removidas e o arquivo consolidado foi exportado como Arquivo5_Final_Filtrado.csv.

Figura 12 - Limpeza de Texto, Padronização de Datas e Regra de Classificação 'Ativa/Inativa'



```
import pandas as pd
from google.colab import drive
drive.mount('/content/drive')
o = '/content/drive/My Drive/TESTE/'
```

```
Mounted at /content/drive
```

```
Arquivo5 = pd.read_csv(o + 'Arquivo5_corretoV2.csv', low_memory=False, sep=';', encoding='UTF-8')
```

```
pd.set_option('display.max_columns', None)
Arquivo5.head(10
              )
```

Mostrar saída oculta

```
Arquivo5 = Arquivo5.map(
    lambda x: ' '.join(x.split()) if isinstance(x, str) else x
)

Arquivo5 = Arquivo5.fillna('Sem Informação')

Arquivo5 = Arquivo5.replace(r'^\s*$', 'Sem Informação', regex=True)

Arquivo5 = Arquivo5.replace(r'^\s*-\s*$', 'Sem Informação', regex=True)
```

```
pd.set_option('display.max_columns', None)
Arquivo5.head(5)
```

Mostrar saída oculta

```
Arquivo5['Inicial'] = pd.to_datetime(
    Arquivo5['Inicial'], dayfirst=True, errors='coerce'
).dt.strftime('%d/%m/%Y')

Arquivo5['Final'] = pd.to_datetime(
    Arquivo5['Final'], dayfirst=True, errors='coerce'
).dt.strftime('%d/%m/%Y')

Arquivo5 = Arquivo5.fillna('Sem Informação')
```

```
pd.set_option('display.max_columns', None)
Arquivo5.head(5)
```

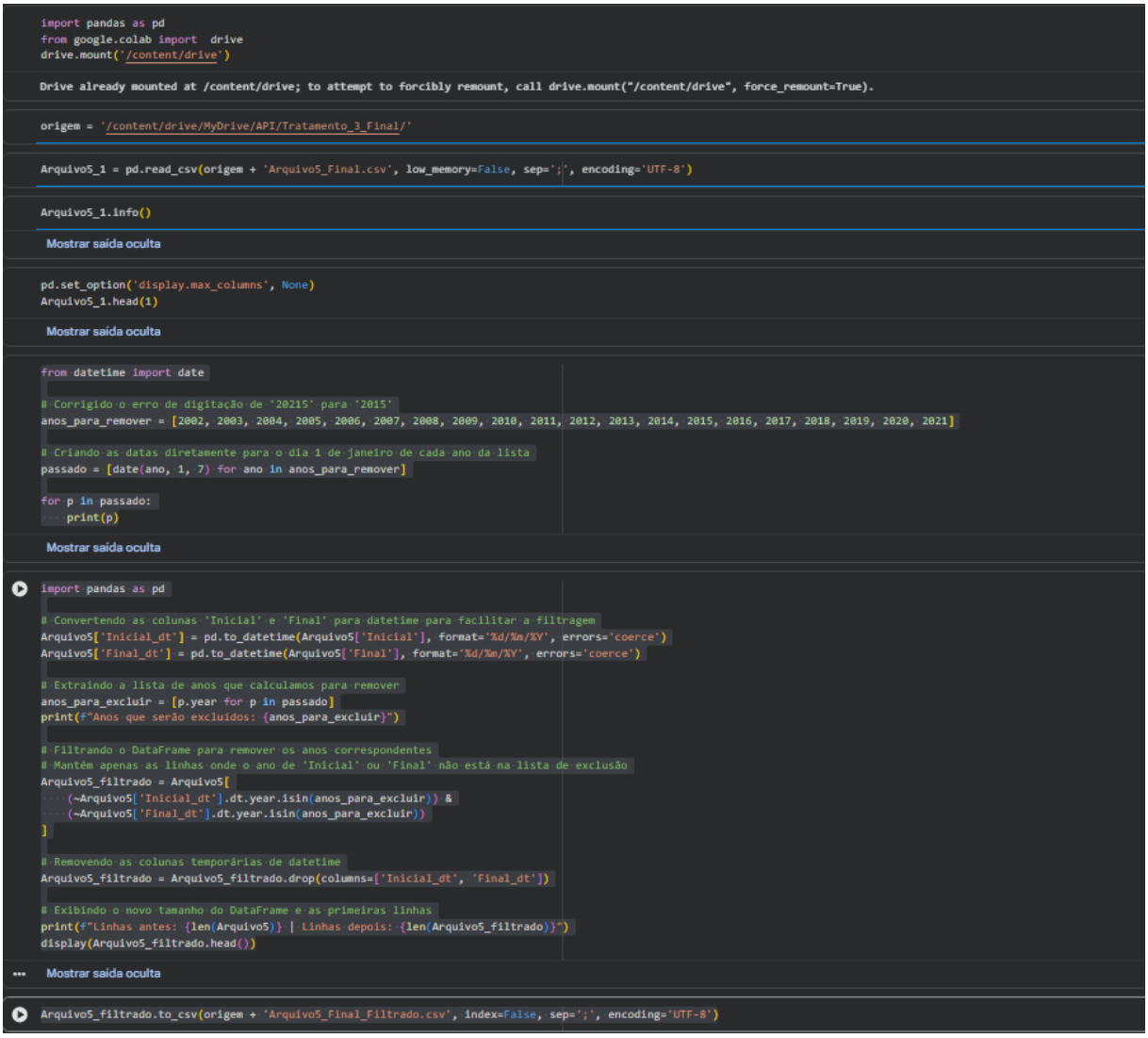
Mostrar saída oculta

```
Arquivo5['Atividade'] = Arquivo5['Linha'].apply(
    lambda x: 'Inativa' if 'C099-Inativos' in str(x) else 'Ativa'
)
```

```
Arquivo5.to_csv(o + 'lucass.csv', index=False, sep=';', encoding='utf-8-sig')

print("Arquivo salvo como arquivo1_correto.csv!")
```

Figura 13 - Conversão de Datas e Filtragem de Exclusão do Histórico de 2002 a 2021



```
import pandas as pd
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
origem = '/content/drive/MyDrive/API/Tratamento_3_Final/'
```

```
Arquivo5_1 = pd.read_csv(origem + 'Arquivo5_Final.csv', low_memory=False, sep=';', encoding='UTF-8')
```

```
Arquivo5_1.info()
```

```
Mostrar saída oculta
```

```
pd.set_option('display.max_columns', None)
Arquivo5_1.head(1)
```

```
Mostrar saída oculta
```

```
from datetime import date

# Corrigido o erro de digitação de '20215' para '2015'
anos_para_remover = [2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021]

# Criando as datas diretamente para o dia 1 de janeiro de cada ano da lista
passado = [date(ano, 1, 7) for ano in anos_para_remover]

for p in passado:
    print(p)
```

```
Mostrar saída oculta
```

```
import pandas as pd

# Convertendo as colunas 'Inicial' e 'Final' para datetime para facilitar a filtragem
Arquivo5['Inicial_dt'] = pd.to_datetime(Arquivo5['Inicial'], format='%d/%m/%Y', errors='coerce')
Arquivo5['Final_dt'] = pd.to_datetime(Arquivo5['Final'], format='%d/%m/%Y', errors='coerce')

# Extraindo a lista de anos que calculamos para remover
anos_para_excluir = [p.year for p in passado]
print(f"Anos que serão excluídos: {anos_para_excluir}")

# Filtrando o DataFrame para remover os anos correspondentes
# Mantém apenas as linhas onde o ano de 'Inicial' ou 'Final' não está na lista de exclusão
Arquivo5_filtrado = Arquivo5[
    (~Arquivo5['Inicial_dt'].dt.year.isin(anos_para_excluir)) &
    (~Arquivo5['Final_dt'].dt.year.isin(anos_para_excluir))
]

# Removendo as colunas temporárias de datetime
Arquivo5_filtrado = Arquivo5_filtrado.drop(columns=['Inicial_dt', 'Final_dt'])

# Exibindo o novo tamanho do DataFrame e as primeiras linhas
print(f"Linhas antes: {len(Arquivo5)} | Linhas depois: {len(Arquivo5_filtrado)}")
display(Arquivo5_filtrado.head())
```

```
Mostrar saída oculta
```

```
Arquivo5_filtrado.to_csv(origem + 'Arquivo5_Final_Filtrado.csv', index=False, sep=';', encoding='UTF-8')
```

4.1.4. Padronização de Unidades de Medida, Saldos e Filtragem Histórica do RES34

O quarto e último bloco da etapa de Engenharia de Dados tratou o relatório RES34 (relatório de estoque atual por almoxarifado do ERP ALVO), responsável por fornecer a fotografia das quantidades físicas, locais de armazenagem, valores unitários e datas da última entrada e saída de cada material. Por ser uma base extensa e com diversidade de preenchimentos operacionais, o tratamento foi dividido em três scripts no Google Colab:

1. Padronização do Dicionário de Unidades de Medida

- **Identificação de Inconsistências:** A planilha arquivo6_corretoV2.csv foi carregada no ambiente. Com os comandos .nunique() e .unique(), a equipe mapeou a existência de dezenas de variações ortográficas e abreviações para as mesmas unidades de medida (como 'pç', 'unid', 'un*', 'l', 'mt').
- **Mapeamento via Dicionário:** Foi estruturado um dicionário de padronização em Python (padronizacao) para mapear e converter todas as siglas em minúsculo ou com caracteres especiais para as siglas oficiais em caixa alta (ex: 'UN', 'L', 'M', 'M²', 'KG', 'CX', 'PCT').
- **Substituição e Exportação Padrão:** A coluna 'UN' foi atualizada através do método .replace(padronizacao) e a base padronizada foi salva como arquivo6_padrão.csv.

2. Higienização de Texto e Conversão Numérica de Saldos e Valores

- **Tratamento de Texto e Nulos:** O script recarrega a base e aplica uma função lambda para eliminar espaços duplicados e quebras de linha nas células de texto. Registros vazios, hífens ou nulos foram preenchidos com o termo 'Sem Informação'.
- **Ajuste Decimal de Saldos e Valores Unitários:** As colunas financeiras e quantitativas 'Saldo Atual' e 'Valor Unit' apresentavam inconsistências na separação de decimais. O script substituiu vírgulas por pontos, aplicou a conversão para tipo numérico real via pd.to_numeric(errors='coerce') e fixou o arredondamento em 2 casas decimais (.round(2)), exportando o resultado padronizado.
- **Filtragem Lógica e Exportação da Base Limpa:** O dataset foi filtrado mantendo apenas as linhas cujas datas de início e término não pertencessem ao intervalo de anos excluídos. Por fim, as colunas temporárias foram removidas e o arquivo consolidado foi exportado como Arquivo5_Final_Filtrado.csv.

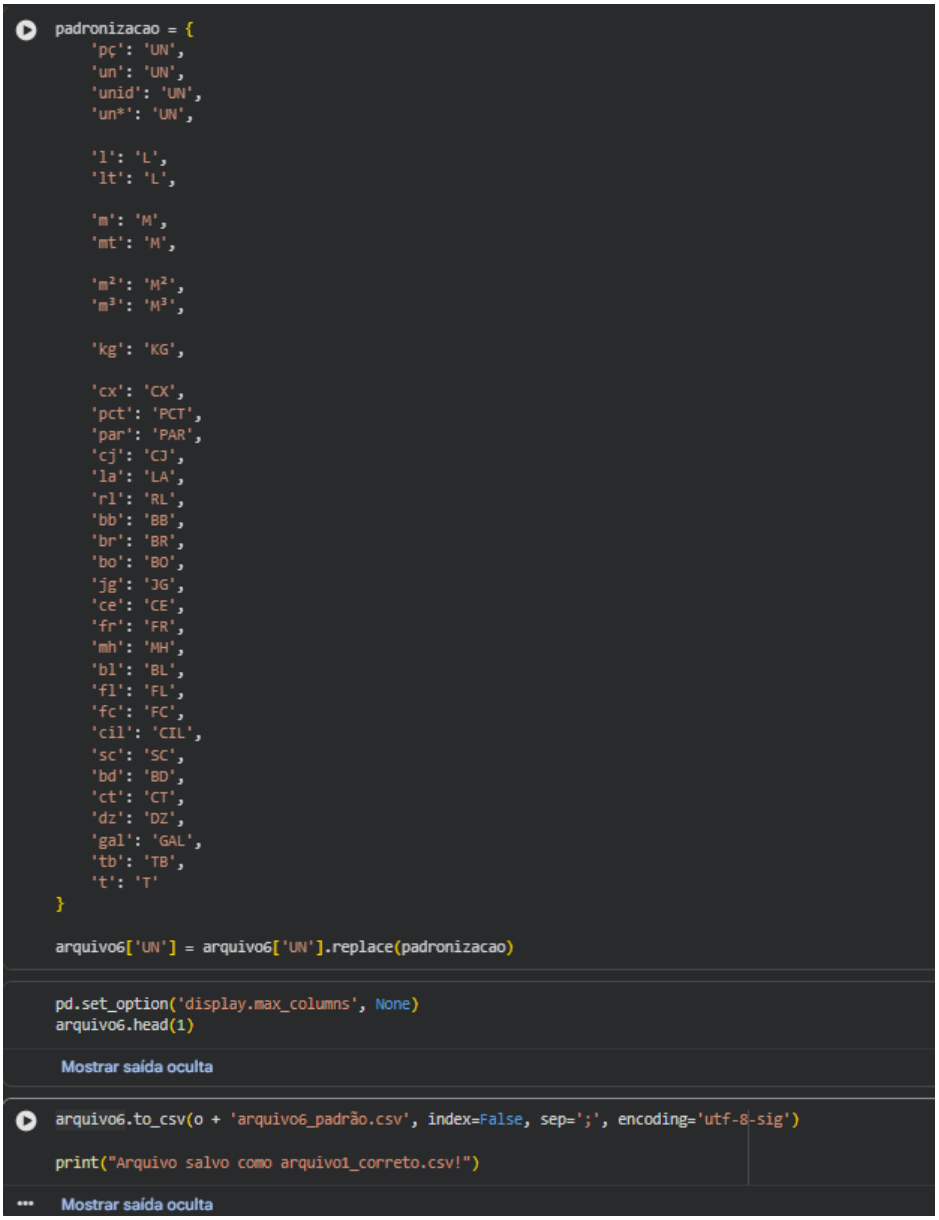
3. Filtragem Histórica por Última Entrada e Última Saí

- **Conversão de Datas Operacionais:** No terceiro script (Arquivo6_Final.csv), as colunas 'Ult. Entrada' e 'Ult. Saída' foram convertidas temporariamente para objetos do tipo datetime ('Inicial_dt' e 'Final_dt') com tratamento de erros.
- **Exclusão de Registros Obsoletos (2002 a 2021):** Para alinhar a base de saldos ao recorte temporal do projeto, aplicou-se uma regra de filtragem lógica que identificou e removeu os registros cujos anos de última movimentação estivessem no intervalo de 2002 a 2021.
- **Geração do Dataset Consolidado do Estoque Atual:** Após a exclusão dos anos antigos e a remoção das colunas temporárias de data, o dataset filtrado foi exportado como Arquivo6_Final_Filtrado.csv, servindo de fonte primária para a determinação do Valor Total Imobilizado e cálculo do Giro de Estoque no Power BI e no site CPTM Insight Flow.

Figura 14 - Dicionário de Padronização das Unidades de Medida ('UN')



Figura 15 - Dicionário de Padronização das Unidades de Medida ('UN') Parte 2



```
padronizacao = {
    'pç': 'UN',
    'un': 'UN',
    'unid': 'UN',
    'un*': 'UN',

    'l': 'L',
    'lt': 'L',

    'm': 'M',
    'mt': 'M',

    'm²': 'M²',
    'm³': 'M³',

    'kg': 'KG',

    'cx': 'CX',
    'pct': 'PCT',
    'par': 'PAR',
    'cj': 'CJ',
    'la': 'LA',
    'rl': 'RL',
    'bb': 'BB',
    'br': 'BR',
    'bo': 'BO',
    'jg': 'JG',
    'ce': 'CE',
    'fr': 'FR',
    'mh': 'MH',
    'bl': 'BL',
    'fl': 'FL',
    'fc': 'FC',
    'cil': 'CIL',
    'sc': 'SC',
    'bd': 'BD',
    'ct': 'CT',
    'dz': 'DZ',
    'gal': 'GAL',
    'tb': 'TB',
    't': 'T'
}

arquivo6['UN'] = arquivo6['UN'].replace(padronizacao)
```

```
pd.set_option('display.max_columns', None)
arquivo6.head(1)
```

Mostrar saída oculta

```
arquivo6.to_csv(o + 'arquivo6_padrão.csv', index=False, sep=';', encoding='utf-8-sig')

print("Arquivo salvo como arquivo1_correto.csv!")
```

Mostrar saída oculta

Figura 16 - Higienização de Strings, Nulos e Conversão Decimal de 'Saldo Atual' e 'Valor Unit'

```
import pandas as pd
from google.colab import drive
drive.mount('/content/drive')
o = '/content/drive/My Drive/TESTE/'
```

```
Arquivo6 = pd.read_csv(o + 'arquivo6.csv', low_memory=False, sep=';', encoding='UTF-8')
```

```
pd.set_option('display.max_columns', None)
Arquivo6.head(1)
```

```
Arquivo6 = Arquivo6.map(
    lambda x: ' '.join(x.split()) if isinstance(x, str) else x
)

Arquivo6 = Arquivo6.fillna('Sem Informação')

Arquivo6 = Arquivo6.replace(r'^\s*$', 'Sem Informação', regex=True)

Arquivo6 = Arquivo6.replace(r'^\s*-\s*$', 'Sem Informação', regex=True)
```

```
pd.set_option('display.max_columns', None)
Arquivo6.head(1)
```

```
colunas = ['Saldo Atual', 'Valor Unit']
for coluna in colunas:Arquivo6[coluna] = (Arquivo6[coluna].astype(str).str.replace('.', ',', regex=False))
```

```
colunas = ['Saldo Atual', 'Valor Unit']

for coluna in colunas:
    Arquivo6[coluna] = pd.to_numeric(
        Arquivo6[coluna].astype(str).str.replace(',', '.', regex=False),
        errors='coerce'
    ).round(2)
```

```
colunas = ['Saldo Atual', 'Valor Unit']

for coluna in colunas:
    Arquivo6[coluna] = (
        pd.to_numeric(
            Arquivo6[coluna].astype(str).str.replace(',', '.', regex=False),
            errors='coerce'
        )
        .round(2)
        .map(lambda x: str(x).replace('.', ',') if pd.notna(x) else x)
    )
```

```
Arquivo6.to_csv(o + 'arquivoultimo.csv', index=False, sep=';', encoding='utf-8-sig')

print("Arquivo salvo como arquivo1_correto.csv!")
```

Figura 17 - Conversão de Datas de Entrada/Saída e Filtragem do Histórico de 2002 a 2021

```
import pandas as pd
from google.colab import  drive
drive.mount('/content/drive')
```

Mostrar saída oculta

```
origem = '/content/drive/MyDrive/API/Tratamento_3_Final/'
```

```
novo = origem + 'Arquivo6_Final.csv'
```

```
# Carrega o arquivo CSV especificando o ponto e vírgula como separador
novo1 = pd.read_csv(novo, sep=';')

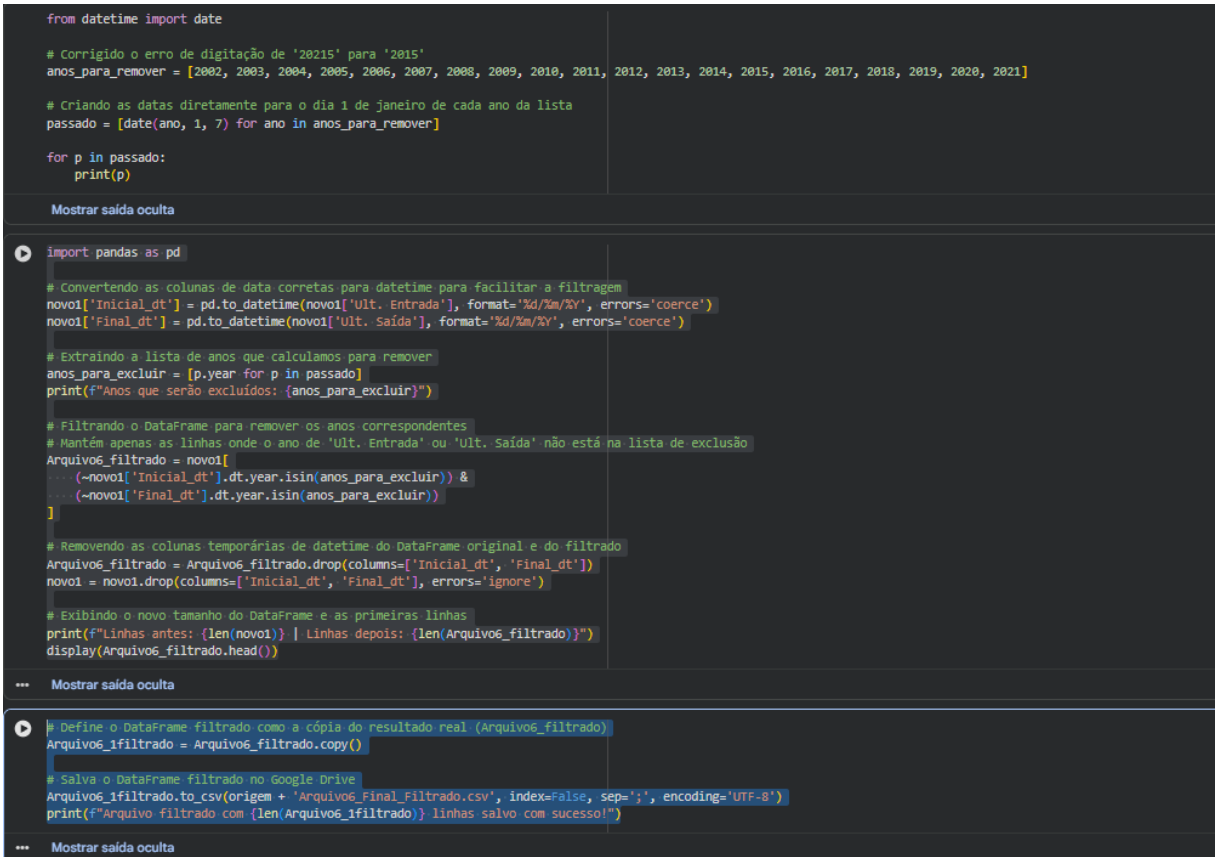
# Exibe as informações do DataFrame
novo1.info()
```

Mostrar saída oculta

```
pd.set_option('display.max_columns', None)
novo1.head(1)
```

Mostrar saída oculta

Figura 18 - Conversão de Datas de Entrada/Saída e Filtragem do Histórico de 2002 a 2021 Parte 2



```
from datetime import date

# Corrigido o erro de digitação de '20215' para '2015'
anos_para_remover = [2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021]

# Criando as datas diretamente para o dia 1 de janeiro de cada ano da lista
passado = [date(ano, 1, 7) for ano in anos_para_remover]

for p in passado:
    print(p)
```

Mostrar saída oculta

```
import pandas as pd

# Convertendo as colunas de data corretas para datetime para facilitar a filtragem
novo1['Inicial_dt'] = pd.to_datetime(novo1['Ult. Entrada'], format='%d/%m/%Y', errors='coerce')
novo1['Final_dt'] = pd.to_datetime(novo1['Ult. Saída'], format='%d/%m/%Y', errors='coerce')

# Extraindo a lista de anos que calculamos para remover
anos_para_excluir = [p.year for p in passado]
print(f"Anos que serão excluídos: {anos_para_excluir}")

# Filtrando o DataFrame para remover os anos correspondentes
# Mantém apenas as linhas onde o ano de 'Ult. Entrada' ou 'Ult. Saída' não está na lista de exclusão
Arquivo6_filtrado = novo1[
    (~novo1['Inicial_dt'].dt.year.isin(anos_para_excluir)) &
    (~novo1['Final_dt'].dt.year.isin(anos_para_excluir))
]

# Removendo as colunas temporárias de datetime do DataFrame original e do filtrado
Arquivo6_filtrado = Arquivo6_filtrado.drop(columns=['Inicial_dt', 'Final_dt'])
novo1 = novo1.drop(columns=['Inicial_dt', 'Final_dt'], errors='ignore')

# Exibindo o novo tamanho do DataFrame e as primeiras linhas
print(f"Linhas antes: {len(novo1)} | Linhas depois: {len(Arquivo6_filtrado)}")
display(Arquivo6_filtrado.head())
```

Mostrar saída oculta

```
# Define o DataFrame filtrado como a cópia do resultado real (Arquivo6_filtrado)
Arquivo6_1filtrado = Arquivo6_filtrado.copy()

# Salva o DataFrame filtrado no Google Drive
Arquivo6_1filtrado.to_csv(origem + 'Arquivo6_Final_Filtrado.csv', index=False, sep=';', encoding='UTF-8')
print(f"Arquivo filtrado com {len(Arquivo6_1filtrado)} linhas salvo com sucesso!")
```

Mostrar saída oculta

4.1.5. Padronização e Filtragem Histórica do Relatório de Estoque e Consumo

O quinto bloco do processo de Engenharia de Dados tratou o relatório RES47 (relatório de estoque e consumo do ERP ALVO), fundamental para avaliar a valoração financeira imobilizada, custos médios unitários e as datas do último consumo registrado para cada item. O tratamento foi estruturado em três scripts no Google Colab:

1. Padronização do Dicionário de Unidades de Medida

- **Identificação de Inconsistências:** A planilha arquivo7_corretoV2.csv foi importada e inspecionada. O código utilizou os comandos .nunique() e .unique() na coluna 'Unid. Medida' para catalogar todas as siglas e grafias utilizadas nos cadastros do sistema.

- **Mapeamento e Substituição Siglar:** Mapeou-se um dicionário de padronização (padronizacao) para converter termos minúsculos, abreviados ou com caracteres especiais para o padrão em caixa alta (como 'UN', 'L', 'M', 'KG', 'CX', 'PCT').

- **Exportação do Dataset Padrão:** A coluna 'Unid. Medida' foi atualizada com o método .replace(padronizacao) e o arquivo resultante foi salvo como arquivo7_padrão.csv.

2. Conversão Numérica de Custos e Tratamento de Nulos

- **Ajuste Decimal de Valores Financeiros:** As colunas 'Custo Médio' e 'Valor Total' foram convertidas para o tipo numérico real com o comando pd.to_numeric(), garantindo o arredondamento preciso para 2 casas decimais.

- **Preenchimento e Higienização de Ausentes:** Células nulas, vazias ou contendo apenas espaços em branco foram substituídas pelo termo 'Sem Informação' via .fillna() e expressões regulares.
- **Formatação Monetária Final:** O script aplicou uma formatação final garantindo a representação adequada de duas casas decimais com vírgula como separador decimal para visualização nos relatórios, salvando a base tratada.

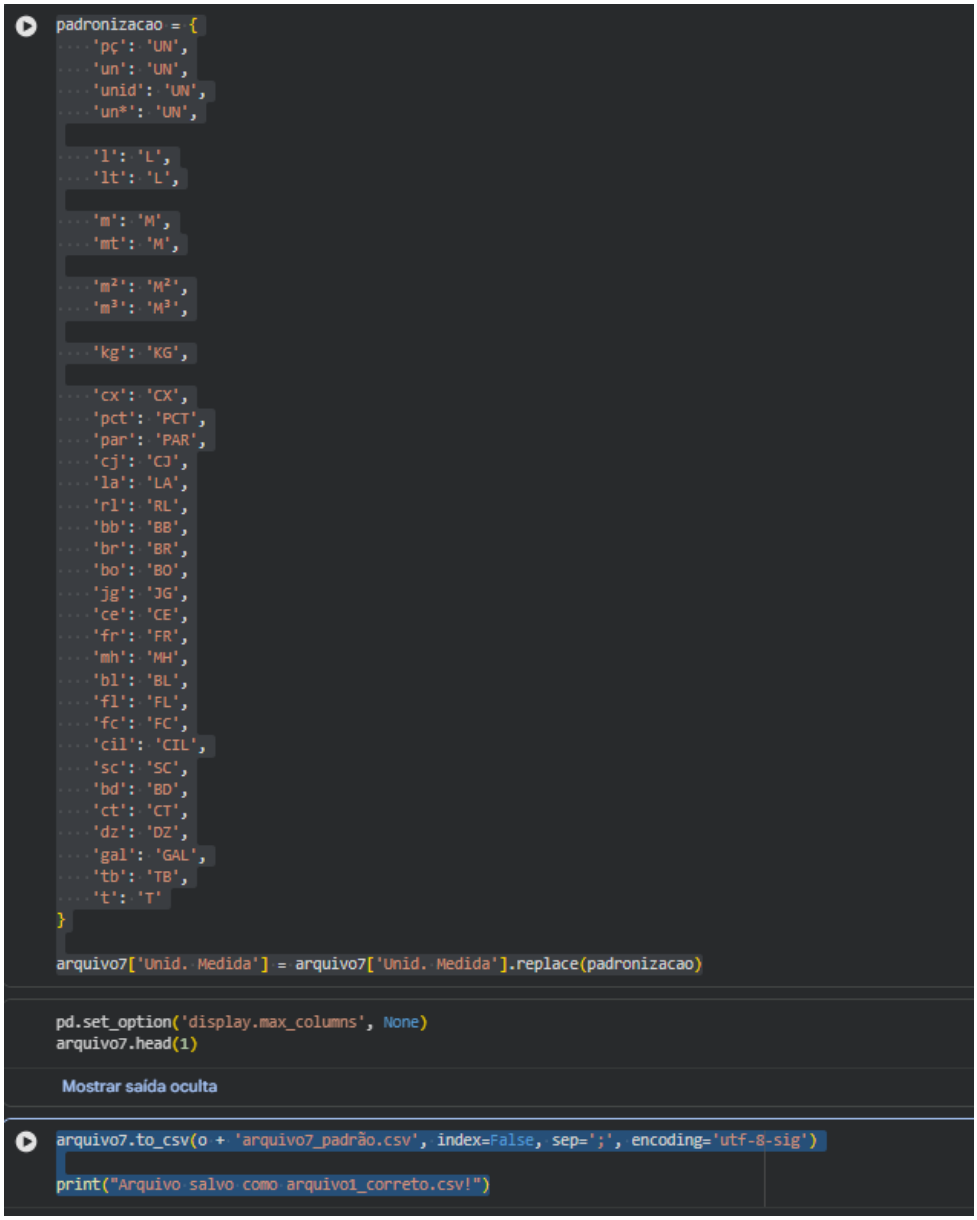
3. Conversão de Datas e Filtragem Temporal Histórica

- **Conversão de Datas de Movimentação e Consumo:** No terceiro script, as colunas 'Última Movimentação' e 'Último Consumo' foram convertidas temporariamente para o formato datetime ('Inicial_dt' e 'Final_dt').
- **Exclusão do Intervalo Histórico Antigo (2002 a 2021):** Para alinhar a base de consumo ao recorte temporal de análise, executou-se a filtragem lógica que identificou e excluiu os registros cujas movimentações pertencem ao período de 2002 a 2021.
- **Geração do Dataset Final RES47:** O dataset filtrado teve suas colunas temporárias removidas e foi exportado como Arquivo7_Final_Filtrado_Exclusivo.csv, fornecendo a base para o cálculo de risco de obsolescência, tempo sem consumo e giro de estoque no Power BI e no site CPTM Insight Flow.

Figura 19 - Mapeamento e Padronização das Unidades de Medida na Coluna 'Unid. Medida'



Figura 20 - Mapeamento e Padronização das Unidades de Medida na Coluna 'Unid. Medida' Parte 2



```
padronizacao = {
    'pç': 'UN',
    'un': 'UN',
    'unid': 'UN',
    'un*': 'UN',

    'l': 'L',
    'lt': 'L',

    'm': 'M',
    'mt': 'M',

    'm²': 'M²',
    'm³': 'M³',

    'kg': 'KG',

    'cx': 'CX',
    'pct': 'PCT',
    'par': 'PAR',
    'cj': 'CJ',
    'la': 'LA',
    'rl': 'RL',
    'bb': 'BB',
    'br': 'BR',
    'bo': 'BO',
    'jg': 'JG',
    'ce': 'CE',
    'fr': 'FR',
    'mh': 'MH',
    'bl': 'BL',
    'fl': 'FL',
    'fc': 'FC',
    'cil': 'CIL',
    'sc': 'SC',
    'bd': 'BD',
    'ct': 'CT',
    'dz': 'DZ',
    'gal': 'GAL',
    'tb': 'TB',
    't': 'T'
}

arquivo7['Unid. Medida'] = arquivo7['Unid. Medida'].replace(padronizacao)
```

```
pd.set_option('display.max_columns', None)
arquivo7.head(1)
```

Mostrar saída oculta

```
arquivo7.to_csv(o + 'arquivo7_padrão.csv', index=False, sep=';', encoding='utf-8-sig')

print("Arquivo salvo como arquivo1_correto.csv!")
```

Figura 21 - Conversão Numérica de 'Custo Médio' e 'Valor Total' e Tratamento de Nulos



```
import pandas as pd
from google.colab import drive
drive.mount('/content/drive')
o = '/content/drive/My Drive/TESTE/'
```

Mostrar saída oculta

```
Arquivo7 = pd.read_csv(o + 'arquivo7_padrão.csv', low_memory=False, sep=';', encoding='UTF-8')
```

```
pd.set_option('display.max_columns', None)
Arquivo7.head(3)
```

Mostrar saída oculta

```
colunas_valor = ['Custo Médio', 'Valor Total']

for coluna in colunas_valor:
    Arquivo7[coluna] = pd.to_numeric(
        Arquivo7[coluna].astype(str).str.replace(',', '.', regex=False),
        errors='coerce'
    ).round(2)
```

```
colunas_valor = ['Custo Médio', 'Valor Total']

for coluna in colunas_valor:
    Arquivo7[coluna] = Arquivo7[coluna].apply(
        lambda x: round(float(x), 2) if str(x).strip() != 'Sem Informação' else x
    )
```

```
pd.set_option('display.max_columns', None)
Arquivo7.head(3)
```

Mostrar saída oculta

```
Arquivo7 = Arquivo7.fillna('Sem Informação')

Arquivo7 = Arquivo7.replace(r'^\s*$', 'Sem Informação', regex=True)
```

```
pd.set_option('display.max_columns', None)
Arquivo7.head(3)
```

Mostrar saída oculta

```
colunas_valor = ['Custo Médio', 'Valor Total']

for coluna in colunas_valor:
    Arquivo7[coluna] = Arquivo7[coluna].apply(
        lambda x: f'{float(x):.2f}'.replace('.', ',')
        if str(x).strip() != 'Sem Informação' else x
    )
```

```
Arquivo7.to_csv(o + 'tudo.csv', index=False, sep=';', encoding='utf-8-sig')

print("Arquivo salvo como arquivo1_correto.csv!")
```

Mostrar saída oculta

Figura 22 - Conversão de Datas de Movimentação/Consumo e Filtragem Histórica (2002 a 2021)



```
import pandas as pd
```

```
from google.colab import  drive
```

```
drive.mount('/content/drive')
```

Mostrar saída oculta

```
origem = '/content/drive/MyDrive/API/Tratamento_3_Final/'
```

```
novo = origem + 'Arquivo7_Final.csv'
```

```
novo1 = pd.read_csv(novo, sep=';')
novo1.info()
```

Mostrar saída oculta

```
pd.set_option('display.max_columns', None)
novo1.head(1)
```

Mostrar saída oculta

```
from datetime import date

# Corrigido o erro de digitação de '20215' para '2015'
anos_para_remover = [2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021]

# Criando as datas diretamente para o dia 1 de janeiro de cada ano da lista
passado = [date(ano, 1, 7) for ano in anos_para_remover]

for p in passado:
    print(p)
```

Mostrar saída oculta

Figura 23 - Conversão de Datas de Movimentação/Consumo e Filtragem Histórica (2002 a 2021) Parte 2



```
import pandas as pd

# Convertendo as colunas de data corretas para datetime para facilitar a filtragem
novo1['Inicial_dt'] = pd.to_datetime(novo1['Última Movimentação'], format='%d/%m/%Y', errors='coerce')
novo1['Final_dt'] = pd.to_datetime(novo1['Último Consumo'], format='%d/%m/%Y', errors='coerce')

# Extraindo a lista de anos que calculamos para remover
anos_para_excluir = [p.year for p in passado]
print(f"Anos que serão excluídos: {anos_para_excluir}")

# Filtrando o DataFrame para remover os anos correspondentes
Arquivo7_filtrado = novo1[
    (~novo1['Inicial_dt'].dt.year.isin(anos_para_excluir)) &
    (~novo1['Final_dt'].dt.year.isin(anos_para_excluir))
].copy()

# Removendo as colunas temporárias de datetime do DataFrame original e do filtrado
Arquivo7_filtrado = Arquivo7_filtrado.drop(columns=['Inicial_dt', 'Final_dt'], errors='ignore')
novo1 = novo1.drop(columns=['Inicial_dt', 'Final_dt'], errors='ignore')

# Exibindo o novo tamanho do DataFrame e as primeiras linhas
print(f"Linhas antes: {len(novo1)} | Linhas depois: {len(Arquivo7_filtrado)}")
display(Arquivo7_filtrado.head())
```

Mostrar saída oculta

```
import pandas as pd

# Criando a coluna temporária de datetime para a coluna 'Última Movimentação'
novo1['Inicial_dt'] = pd.to_datetime(novo1['Última Movimentação'], format='%d/%m/%Y', errors='coerce')

# Filtrando para remover os anos desejados apenas dessa coluna
Arquivo7_filtrado_exclusivo = novo1[
    ~novo1['Inicial_dt'].dt.year.isin(anos_para_excluir)
].copy()

# Removendo a coluna temporária do DataFrame filtrado e do original
Arquivo7_filtrado_exclusivo = Arquivo7_filtrado_exclusivo.drop(columns=['Inicial_dt'], errors='ignore')
novo1 = novo1.drop(columns=['Inicial_dt'], errors='ignore')

# Exibindo o novo tamanho após filtrar apenas pela Última Movimentação
print(f"Linhas antes: {len(novo1)} | Linhas depois (Filtro exclusivo na Última Movimentação): {len(Arquivo7_filtrado_exclusivo)}")

# Salvando o resultado filtrado no Drive
Arquivo7_filtrado_exclusivo.to_csv(origem + 'Arquivo7_Final_Filtrado_Exclusivo.csv', index=False, sep=';', encoding='UTF-8')
print("Arquivo salvo com sucesso com o novo filtro!")
display(Arquivo7_filtrado_exclusivo.head())
```

Mostrar saída oculta

```
# Define o DataFrame filtrado como a cópia do resultado real (Arquivo6_filtrado)
Arquivo7_1filtrado = Arquivo7_1filtrado.copy()

# Salva o DataFrame filtrado no Google Drive
Arquivo7_1filtrado.to_csv(origem + 'Arquivo6_Final_Filtrado.csv', index=False, sep=';', encoding='UTF-8')
print(f"Arquivo filtrado com {len(Arquivo7_1filtrado)} linhas salvo com sucesso!")
```

4.1.6. Padronização, Higienização e Filtragem Histórica do RES75

O sexto bloco do processo de Engenharia de Dados tratou o relatório RES75 (relatório de consumo de materiais por unidade organizacional do ERP ALVO). Esta base é estratégica para comparar o consumo real entregue pelos almoxarifados (MOV) com a demanda orçada e aprovada pelas áreas (PCA). O tratamento foi executado em três scripts no Google Colab:

1. Padronização do Dicionário de Unidades de Medida

- **Identificação de Inconsistências:** A planilha arquivo8_corretoV2.csv foi importada e inspecionada. Por meio dos comandos .nunique() e .unique() aplicados à coluna 'Unid. Medida Principal', mapeou-se a diversidade de siglas cadastradas.
- **Mapeamento e Substituição Siglar:** Construiu-se um dicionário de substituição em Python (padronizacao) para converter todas as siglas abreviadas ou minúsculas para o padrão oficial em caixa alta (como 'UN', 'L', 'M', 'KG', 'CX', 'KIT', 'ML').
- **Exportação do Dataset Padrão:** A coluna 'Unid. Medida Principal' foi atualizada com o método .replace(padronizacao) e o arquivo padronizado foi exportado como arquivo8_padrão.csv.

2. Limpeza de Estrutura, Remoção de Linha Nula e Higienização

- **Ajuste de Cabeçalho e Índices:** Na segunda etapa, o script elimina o primeiro registro redundante da tabela com o comando .drop(index=0) e reorienta os índices da base com .reset_index(drop=True).
- **Tratamento de Strings e Ausentes:** Espaços em branco sobressalentes foram removidos das células de texto. Células nulas, vazias ou preenchidas com marcadores em branco foram padronizadas para o termo 'Sem Informação' via .fillna() e expressões regulares, salvando a base limpa.

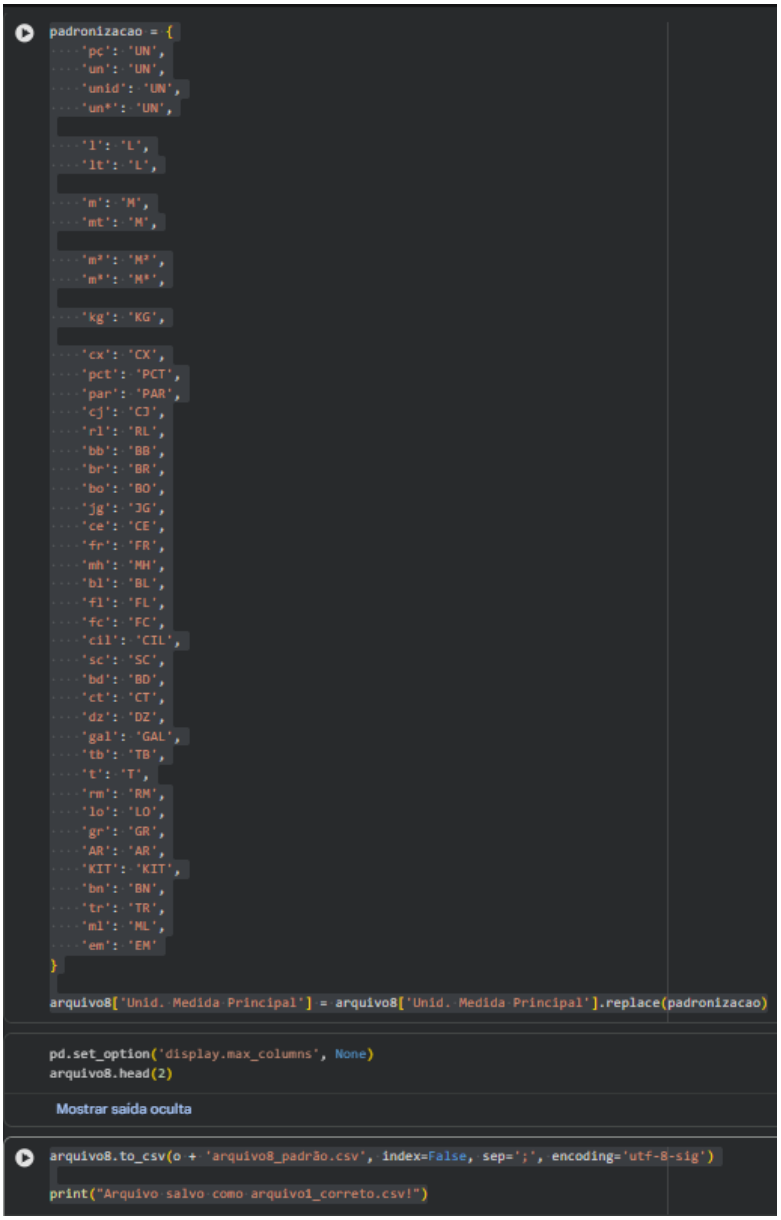
3. Conversão de Datas e Filtragem Temporal Histórica

- **Conversão de Datas com Tratamento de Exceções:** No terceiro script, as colunas de período 'Inicial' e 'Final' foram convertidas para o formato datetime ('Inicial_dt' e 'Final_dt'). O código inclui um bloco de controle try/except KeyError para garantir a integridade da execução e validar a existência das colunas no dataset.
- **Exclusão do Intervalo Histórico Antigo (2002 a 2021):** Aplicou-se a regra de filtragem temporal que identificou e removeu os registros pertencentes ao período de 2002 a 2021 (anos_para_excluir).
- **Geração do Dataset Consolidado do RES75:** As colunas temporárias de data foram descartadas e a base tratada foi exportada como Arquivo8_Final_Filtrado.csv. O arquivo viabiliza as análises de aderência entre planejamento (PCA) e consumo real (MOV) no Power BI e na aplicação web CPTM Insight Flow.

Figura 24 - Dicionário e Padronização das Unidades de Medida na Coluna 'Unid. Medida Principal'



Figura 25 - Dicionário e Padronização das Unidades de Medida na Coluna 'Unid. Medida Principal' Parte 2



```
padronizacao = {
    'pc': 'UN',
    'un': 'UN',
    'unid': 'UN',
    'un*': 'UN',

    'l': 'L',
    'lt': 'L',

    'm': 'M',
    'mt': 'M',

    'm²': 'M²',
    'm*': 'M*',

    'kg': 'KG',

    'cx': 'CX',
    'pct': 'PCT',
    'par': 'PAR',
    'cj': 'CJ',
    'rl': 'RL',
    'bb': 'BB',
    'br': 'BR',
    'bo': 'BO',
    'jg': 'JG',
    'ce': 'CE',
    'fr': 'FR',
    'mh': 'MH',
    'bl': 'BL',
    'fl': 'FL',
    'fc': 'FC',
    'cil': 'CIL',
    'sc': 'SC',
    'bd': 'BD',
    'ct': 'CT',
    'dz': 'DZ',
    'gal': 'GAL',
    'tb': 'TB',
    't': 'T',
    'rm': 'RM',
    'lo': 'LO',
    'gr': 'GR',
    'AR': 'AR',
    'KIT': 'KIT',
    'bn': 'BN',
    'tr': 'TR',
    'ml': 'ML',
    'em': 'EM'
}
arquivo8['Unid. Medida Principal'] = arquivo8['Unid. Medida Principal'].replace(padronizacao)

pd.set_option('display.max_columns', None)
arquivo8.head(2)

Mostrar saida oculta

arquivo8.to_csv(o + 'arquivo8_padrão.csv', index=False, sep=';', encoding='utf-8-sig')

print("Arquivo salvo como arquivo1_correto.csv!")
```

Figura 26 - Remoção de Linha Redundante (drop index=0), Reset de Índice e Preenchimento de Nulos

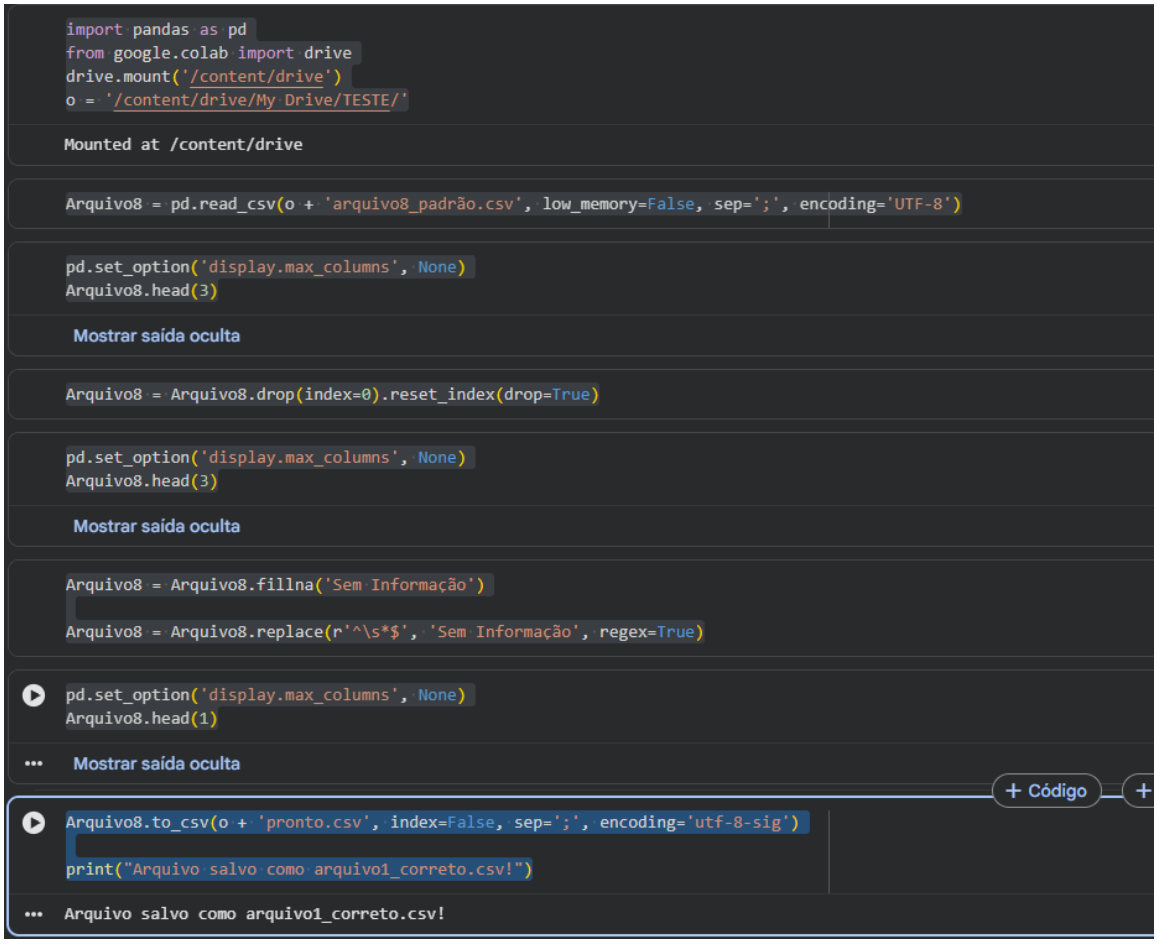
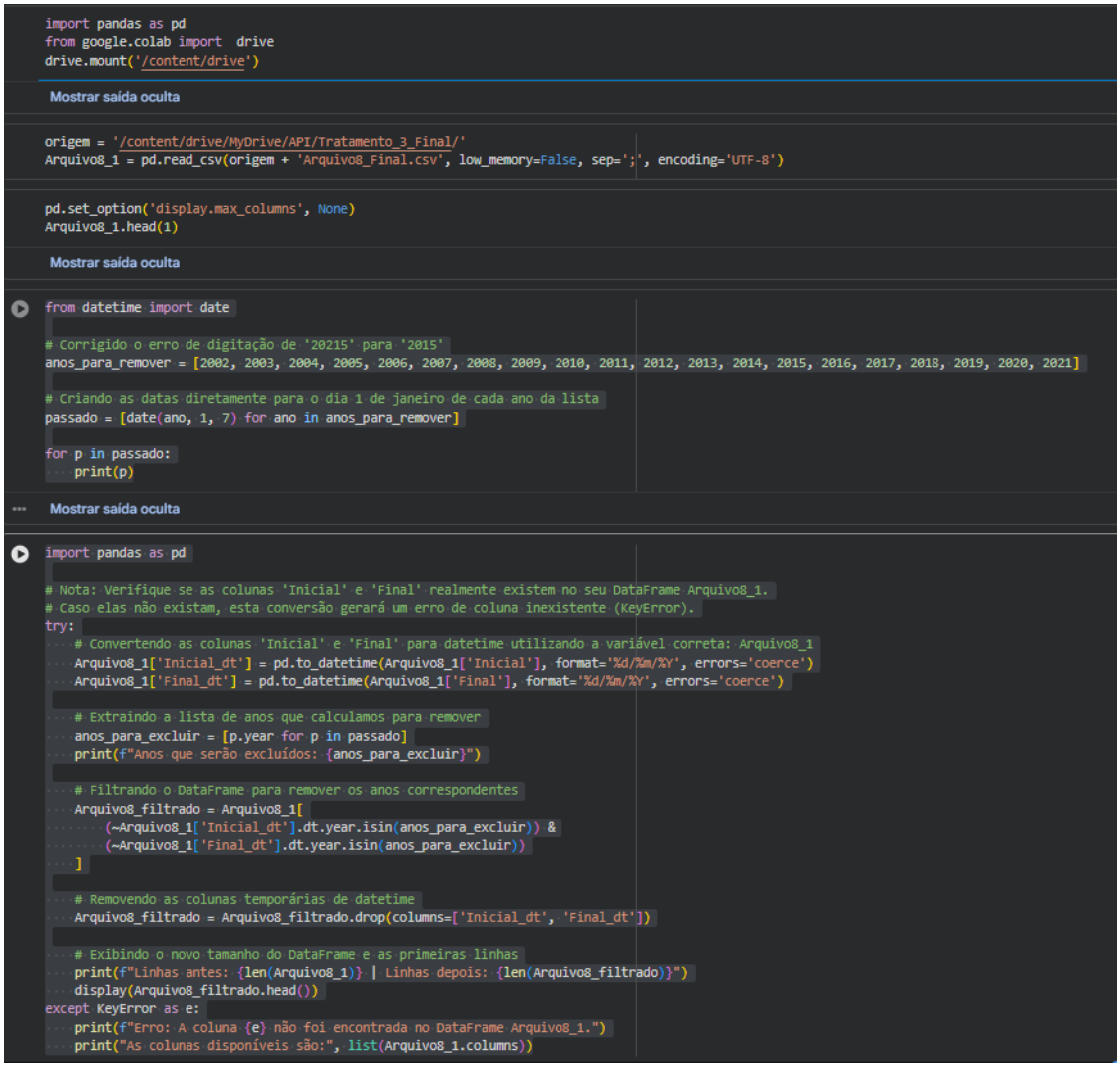


Figura 27 - Conversão de Datas com Tratamento de Exceção (try/except) e Filtragem Histórica (2002 a 2021)



```
import pandas as pd
from google.colab import drive
drive.mount('/content/drive')
```

Mostrar saída oculta

```
origem = '/content/drive/MyDrive/API/Tratamento_3_Final/'
Arquivo8_1 = pd.read_csv(origem + 'Arquivo8_Final.csv', low_memory=False, sep=';', encoding='UTF-8')

pd.set_option('display.max_columns', None)
Arquivo8_1.head(1)
```

Mostrar saída oculta

```
from datetime import date

# Corrigido o erro de digitação de '20215' para '2015'
anos_para_remover = [2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021]

# Criando as datas diretamente para o dia 1 de janeiro de cada ano da lista
passado = [date(ano, 1, 7) for ano in anos_para_remover]

for p in passado:
    print(p)
```

Mostrar saída oculta

```
import pandas as pd

# Nota: Verifique se as colunas 'Inicial' e 'Final' realmente existem no seu DataFrame Arquivo8_1.
# Caso elas não existam, esta conversão gerará um erro de coluna inexistente (KeyError).
try:
    # Convertendo as colunas 'Inicial' e 'Final' para datetime utilizando a variável correta: Arquivo8_1
    Arquivo8_1['Inicial_dt'] = pd.to_datetime(Arquivo8_1['Inicial'], format='%d/%m/%Y', errors='coerce')
    Arquivo8_1['Final_dt'] = pd.to_datetime(Arquivo8_1['Final'], format='%d/%m/%Y', errors='coerce')

    # Extraindo a lista de anos que calculamos para remover
    anos_para_excluir = [p.year for p in passado]
    print(f"Anos que serão excluídos: {anos_para_excluir}")

    # Filtrando o DataFrame para remover os anos correspondentes
    Arquivo8_filtrado = Arquivo8_1[
        (~Arquivo8_1['Inicial_dt'].dt.year.isin(anos_para_excluir)) &
        (~Arquivo8_1['Final_dt'].dt.year.isin(anos_para_excluir))
    ]

    # Removendo as colunas temporárias de datetime
    Arquivo8_filtrado = Arquivo8_filtrado.drop(columns=['Inicial_dt', 'Final_dt'])

    # Exibindo o novo tamanho do DataFrame e as primeiras linhas
    print(f"Linhas antes: {len(Arquivo8_1)} | Linhas depois: {len(Arquivo8_filtrado)}")
    display(Arquivo8_filtrado.head())
except KeyError as e:
    print(f"Erro: A coluna {e} não foi encontrada no DataFrame Arquivo8_1.")
    print("As colunas disponíveis são:", list(Arquivo8_1.columns))
```

4.1.7. Tratamento, Filtragem Histórica e Desmembramento por Regex do RES82

O sétimo e último bloco do processo de Engenharia de Dados tratou o relatório RES82 (base de movimentações históricas de entrada do ERP ALVO). Este relatório é fundamental para auditoria, rastreabilidade de suprimentos e identificação do histórico de preços unitários praticados pelos fornecedores. O processamento foi executado em quatro scripts no Google Colab:

1. Limpeza Estrutural e Remoção de Coluna Secundária
    - **Carga e Inspeção:** A planilha arquivo9_corretoV2.csv foi importada para o ambiente Colab.
    - **Exclusão de Atributo Irrelevante:** Identificou-se que a coluna 'Complemento' continha apenas observações secundárias sem impacto para os indicadores de estoque. A coluna foi removida com o comando .drop(columns=['Complemento']), e o resultado intermediário foi exportado como arquivo9_padrão.csv.

2. Conversão Numérica do Valor Unitário e Preenchimento de Ausentes
    - **Ajuste Decimal de Preços:** A coluna 'Valor Unit.' passou por higienização com .str.replace(',', '.') e conversão para o tipo numérico real, aplicando-se o arredondamento em 2 casas decimais (.round(2)).

- **Tratamento de Nulos e Formatação:** Registros vazios ou nulos foram substituídos pela string 'Sem Informação' via .fillna(). A base foi salva com padrão de separador decimal e codificação UTF-8.

3. Conversão de Datas de Entrada e Filtragem Histórica
   - **Tratamento Temporal:** A coluna de data 'Entrada' foi convertida para o formato datetime ('Entrada_dt') para permitir operações lógicas de intervalo.
   - **Exclusão do Intervalo Histórico Antigo (2002 a 2021):** O script aplicou o filtro de exclusão dos anos antigos (anos_para_excluir), removendo registros de movimentações obsoletas. O dataset resultante foi exportado como Arquivo9_1_Final_Filtrado.csv.

4. Extração Estruturada do Código e Descrição do Produto via Regex
   - **Desmembramento de Atributos Compostos:** Como a coluna original de produto combinava o código e o nome do item na mesma célula (separados por hífen), aplicou-se o método .str.extract() com a expressão regular r'^\s*(.*?)\s+-\s+(.*)$'.
   - **Criação de Colunas Chave:** A operação separou o texto em duas colunas distintas: 'Código_Produto' e 'Produto', salvando o arquivo final como Arquivo9_separado.csv. Essa estruturação viabiliza a criação de relacionamentos diretos com a tabela dimensão de cadastro (dMaterial) no Power BI e na aplicação web CPTM Insight Flow App.

Figura 28 - Remoção da Coluna 'Complemento' e Carga Inicial do RES82

```
import pandas as pd
from google.colab import drive
drive.mount('/content/drive')
o = '/content/drive/My Drive/API/'
```

Mostrar saída oculta

```
arquivo9 = pd.read_csv(o + 'arquivo9_corretoV2.csv', low_memory=False, sep=';', encoding='UTF-8')
```

```
pd.set_option('display.max_columns', None)
arquivo9.head(1)
```

Mostrar saída oculta

```
colunas_apagar = ['Complemento']
arquivo9 = arquivo9.drop(columns=colunas_apagar)
print("Colunas apagadas com sucesso!")
```

Mostrar saída oculta

```
pd.set_option('display.max_columns', None)
arquivo9.head(1)
```

Mostrar saída oculta

```
arquivo9.to_csv(o + 'arquivo9_padrão.csv', index=False, sep=';', encoding='utf-8-sig')

print("Arquivo salvo como arquivo1_correto.csv!")
```

Mostrar saída oculta

Figura 29 - Tratamento Decimal de 'Valor Unit.' e Preenchimento com 'Sem Informação'

```
import pandas as pd
from google.colab import drive
drive.mount('/content/drive')
o = '/content/drive/My Drive/TESTE/'
```

Mostrar saída oculta

```
Arquivo9 = pd.read_csv(o + 'arquivo9_padrão.csv', low_memory=False, sep=';', encoding='UTF-8')
```

```
pd.set_option('display.max_columns', None)
Arquivo9.head(3)
```

Mostrar saída oculta

```
Arquivo9['Valor Unit.'] = (
    Arquivo9['Valor Unit.']
    .astype(str)
    .str.replace(',', '.', regex=False)
    .astype(float)
    .round(2)
)
```

```
pd.set_option('display.max_columns', None)
Arquivo9.head(3)
```

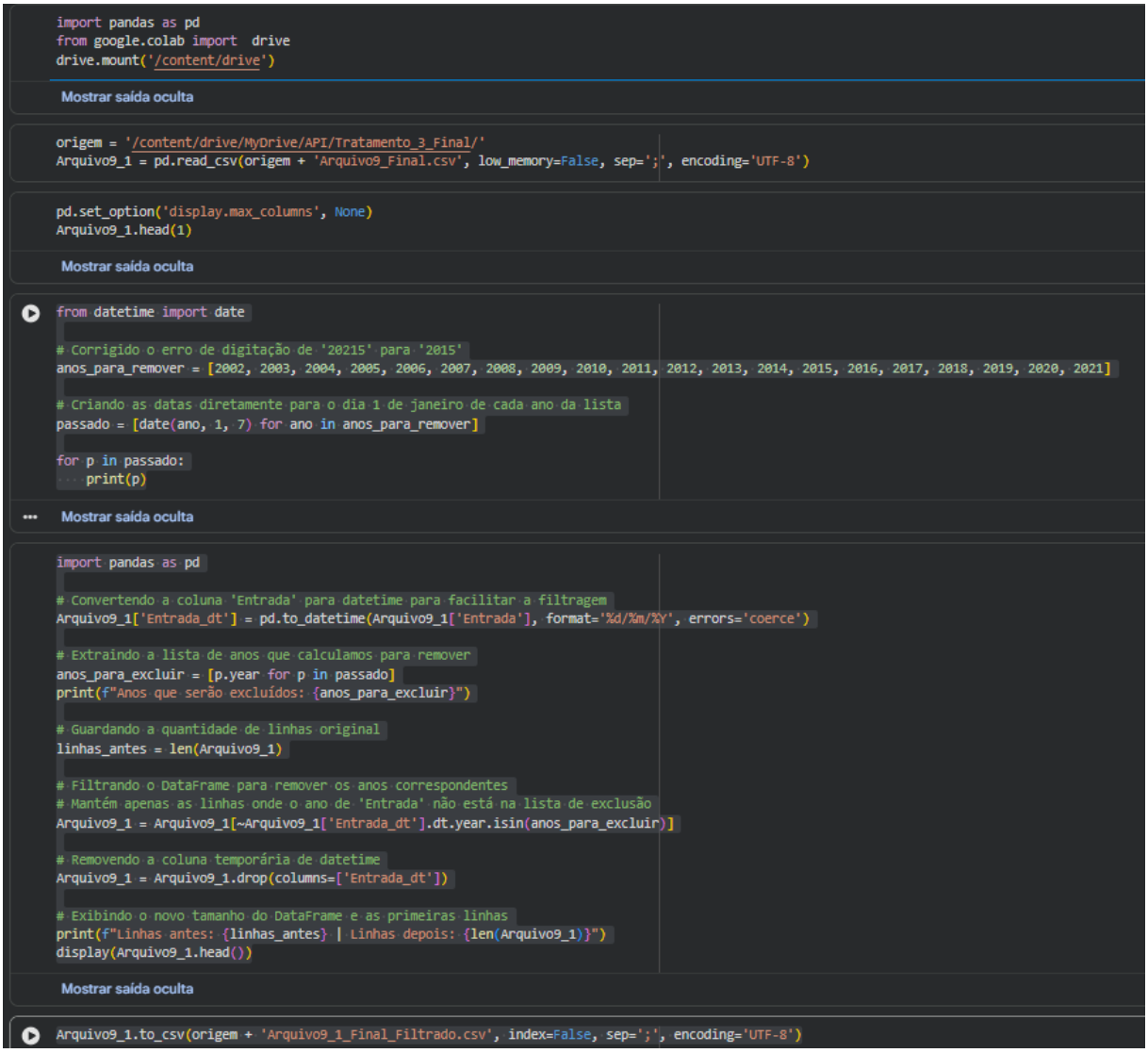
Mostrar saída oculta

```
Arquivo9 = Arquivo9.fillna('Sem Informação')

Arquivo9 = Arquivo9.replace(r'^\s*$', 'Sem Informação', regex=True)
```

```
Arquivo9.to_csv(
    o + 'arquivo9final.csv',
    sep=';',
    index=False,
    encoding='UTF-8',
    decimal=','
)
```

Figura 30 - Conversão de Data de Entrada e Filtragem Histórica (2002 a 2021)



```
import pandas as pd
from google.colab import  drive
drive.mount('/content/drive')
```

Mostrar saída oculta

```
origem = '/content/drive/MyDrive/API/Tratamento_3_Final/'
Arquivo9_1 = pd.read_csv(origem + 'Arquivo9_Final.csv', low_memory=False, sep=';', encoding='UTF-8')
```

```
pd.set_option('display.max_columns', None)
Arquivo9_1.head(1)
```

Mostrar saída oculta

```
from datetime import date

# Corrigido o erro de digitação de '20215' para '2015'
anos_para_remover = [2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021]

# Criando as datas diretamente para o dia 1 de janeiro de cada ano da lista
passado = [date(ano, 1, 7) for ano in anos_para_remover]

for p in passado:
    print(p)
```

Mostrar saída oculta

```
import pandas as pd

# Convertendo a coluna 'Entrada' para datetime para facilitar a filtragem
Arquivo9_1['Entrada_dt'] = pd.to_datetime(Arquivo9_1['Entrada'], format='%d/%m/%Y', errors='coerce')

# Extraindo a lista de anos que calculamos para remover
anos_para_excluir = [p.year for p in passado]
print(f"Anos que serão excluídos: {anos_para_excluir}")

# Guardando a quantidade de linhas original
linhas_antes = len(Arquivo9_1)

# Filtrando o DataFrame para remover os anos correspondentes
# Mantém apenas as linhas onde o ano de 'Entrada' não está na lista de exclusão
Arquivo9_1 = Arquivo9_1[~Arquivo9_1['Entrada_dt'].dt.year.isin(anos_para_excluir)]

# Removendo a coluna temporária de datetime
Arquivo9_1 = Arquivo9_1.drop(columns=['Entrada_dt'])

# Exibindo o novo tamanho do DataFrame e as primeiras linhas
print(f"Linhas antes: {linhas_antes} | Linhas depois: {len(Arquivo9_1)}")
display(Arquivo9_1.head())
```

Mostrar saída oculta

```
Arquivo9_1.to_csv(origem + 'Arquivo9_1_Final_Filtrado.csv', index=False, sep=';', encoding='UTF-8')
```

Figura 31 - Extração via Expressão Regular (Regex) para Separação de 'Código_Produto' e 'Produto'

```
import pandas as pd
from google.colab import drive
drive.mount('/content/drive')
o ='/content/drive/My Drive/Silva/'
```

Mostrar saída oculta

```
Arquivo9 = pd.read_csv(o + 'Arquivo9.csv', low_memory=False, sep=';', encoding='UTF-8')
```

```
Arquivo9.head()
```

Mostrar saída oculta

```
Arquivo9[['Código_Produto', 'Produto']] = (
    Arquivo9['Produto']
    .str.extract(r'^\s*(.*?)\s+-\s+(.*)$')
)


display(Arquivo9[['Código_Produto', 'Produto']].head(10))
```

Mostrar saída oculta

```
Arquivo9.to_csv(
    o + 'Arquivo9_separado.csv',
    index=False,
    sep=';',
    encoding='UTF-8'
)
```

4.1.8. Padronização de Datas Operacionais e Filtragem Histórica do Fluxo de Requisições

O oitavo e último bloco da etapa de Engenharia de Dados tratou a base de Movimentações de Materiais (Arquivo 8), responsável por registrar o ciclo de vida completo das requisições de materiais nos almoxarifados da CPTM (desde a solicitação inicial até a aprovação, separação, conferência, entrega e retirada pelo solicitante). Por acompanhar todas as etapas operacionais e conter múltiplos carimbos de data, o tratamento foi estruturado em três scripts no Google Colab:

1. Limpeza Estrutural e Exclusão de Atributos Irrelevantes
   - **Carga e Inspeção:** A planilha arquivo10_corretoV2.csv foi importada para o ambiente do Colab.
   - **Remoção de Coluna Secundária:** Identificou-se que a coluna 'Equipamento' apresentava altíssima taxa de dados ausentes e não agregava valor aos indicadores de fluxo logístico. A coluna foi excluída com o comando .drop(columns=['Equipamento']), e o arquivo intermediário foi salvo como arquivo10_padrão.csv.

2. Padronização Simultânea de Datas Operacionais e Higienização de Nulos
   - **Padronização em Lote de Colunas Temporais:** A base continha 7 colunas distintas de registro temporal ('Data Aprovação', 'Data Entrega', 'Data Conferência', 'Data Separação', 'Data Fim Separação', 'Data Início Separação' e 'Data'). O script aplicou

um laço de repetição (for) para converter todas essas colunas em datas válidas com a função pd.to_datetime(..., dayfirst=True) e formatá-las no padrão brasileiro 'dd/mm/aaaa'.

- **Tratamento de Células Ausentes:** Registros nulos, vazios ou contendo apenas espaços em branco foram substituídos pela string 'Sem Informação' via .fillna(), gerando uma base homogênea para análise de prazos de atendimento.

3. Conversão de Intervalo e Filtragem Histórica
   - **Mapeamento de Período:** Para viabilizar a filtragem temporal, as colunas 'Data Aprovação' e 'Data Entrega' foram convertidas temporariamente para o formato datetime ('Inicial_dt' e 'Final_dt').
   - **Exclusão do Intervalo Histórico Antigo (2002 a 2021):** Aplicou-se a regra de exclusão temporal (anos_para_excluir) para eliminar movimentações e solicitações obsoletas anteriores a 2022.
   - **Geração do Dataset Consolidado das Movimentações:** As colunas temporárias foram removidas e a base tratada foi exportada como Arquivo10_1_Final_Filtrado.csv. O arquivo final permite auditar o tempo de atendimento dos almoxarifados e avaliar a eficiência do fluxo de entrega e separação de materiais no Power BI e na aplicação web CPTM Insight Flow App.

Figura 32 - Remoção da Coluna 'Equipamento' e Exportação da Base Padrão



Figura 33 - Padronização em Lote de 7 Colunas de Data e Preenchimento de Nulos

```
import pandas as pd
from google.colab import drive
drive.mount('/content/drive')
o = '/content/drive/My Drive/TESTE/'
```

Mostrar saída oculta

```
Arquivo10 = pd.read_csv(o + 'arquivo10_padrão.csv', low_memory=False, sep=';', encoding='UTF-8')
```

```
pd.set_option('display.max_columns', None)
Arquivo10.head(1)
```

Mostrar saída oculta

```
colunas_data = [
    'Data Aprovação',
    'Data Entrega',
    'Data Conferência',
    'Data Separação',
    'Data Fim Separação',
    'Data Início Separação',
    'Data'
]

for coluna in colunas_data:
    Arquivo10[coluna] = pd.to_datetime(
        Arquivo10[coluna],
        dayfirst=True,
        errors='coerce'
    ).dt.strftime('%d/%m/%Y')
```

```
pd.set_option('display.max_columns', None)
Arquivo10.head(1)
```

Mostrar saída oculta

```
Arquivo10 = Arquivo10.fillna('Sem Informação')

Arquivo10 = Arquivo10.replace(r'^\s*$', 'Sem Informação', regex=True)
```

```
Arquivo10.to_csv(o + 'testeN10.csv', index=False, sep=';', encoding='utf-8-sig')

print("Arquivo salvo como arquivo1_correto.csv!")
```

Figura 34 - Conversão de Datas de Aprovação/Entrega e Filtragem Temporal Histórica (2002 a 2021)

```
import pandas as pd
from google.colab import  drive
drive.mount('/content/drive')
```

Mostrar saída oculta

```
origem = '/content/drive/MyDrive/API/Tratamento_3_Final/'
Arquivo10_1 = pd.read_csv(origem + 'Arquivo10_Final.csv', low_memory=False, sep=';', encoding='UTF-8')
```

```
pd.set_option('display.max_columns', None)
Arquivo10_1.head(1)
```

Mostrar saída oculta

```
from datetime import date

# Corrigido o erro de digitação de '20215' para '2015'
anos_para_remover = [2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021]

# Criando as datas diretamente para o dia 1 de janeiro de cada ano da lista
passado = [date(ano, 1, 7) for ano in anos_para_remover]

for p in passado:
    print(p)
```

Mostrar saída oculta

Figura 35 - Conversão de Datas de Aprovação/Entrega e Filtragem Temporal Histórica (2002 a 2021) Parte 2

```python
import pandas as pd

# Substitua 'Data Aprovação' e 'Data Entrega' pelas colunas de data corretas do seu DataFrame
coluna_inicial = 'Data Aprovação'
coluna_final = 'Data Entrega'

# Convertendo as colunas para datetime para facilitar a filtragem
Arquivo10_1['Inicial_dt'] = pd.to_datetime(Arquivo10_1[coluna_inicial], format='%d/%m/%Y', errors='coerce')
Arquivo10_1['Final_dt'] = pd.to_datetime(Arquivo10_1[coluna_final], format='%d/%m/%Y', errors='coerce')

# Extraindo a lista de anos que calculamos para remover
anos_para_excluir = [p.year for p in passado]
print(f"Anos que serão excluídos: {anos_para_excluir}")

tamanho_antes = len(Arquivo10_1)

# Filtrando o DataFrame para remover os anos correspondentes
# Mantém apenas as linhas onde o ano de 'Inicial' ou 'Final' não está na lista de exclusão
Arquivo10_1 = Arquivo10_1[
    (~Arquivo10_1['Inicial_dt'].dt.year.isin(anos_para_excluir)) &
    (~Arquivo10_1['Final_dt'].dt.year.isin(anos_para_excluir))
]

# Removendo as colunas temporárias de datetime
Arquivo10_1 = Arquivo10_1.drop(columns=['Inicial_dt', 'Final_dt'])

# Exibindo o novo tamanho do DataFrame e as primeiras linhas
print(f"Linhas antes: {tamanho_antes} | Linhas depois: {len(Arquivo10_1)}")
display(Arquivo10_1.head())
```

```
Mostrar saída oculta
```

```python
Arquivo10_1.to_csv(origem + 'Arquivo10_1_Final_Filtrado.csv', index=False, sep=';', encoding='UTF-8')
```

4.1.9 Script de Auditoria, Validação e Harmonização Complementar das Bases

Após o tratamento individual e a filtragem primária de cada uma das tabelas do ERP ALVO, desenvolveu-se um script final no Google Colab dedicado exclusivamente à auditoria de integridade, consolidação dos dados e tratamento de bases complementares. Este processo garantiu que todos os datasets estivessem em perfeita concordância estrutural e temporal antes do carregamento final no Power BI e na aplicação web CPTM Insight Flow App.

O fluxo de revisão compreendeu três frentes técnicas de validação:

1. **Carregamento e Verificação Integrada das Bases Finais:** O script realiza a leitura simultânea das oito tabelas consolidadas (Arquivo1_Final.csv a Arquivo10_Final.csv) armazenadas no diretório do Google Drive. Durante a carga, o código valida a integridade das colunas, a consistência dos nomes dos atributos e confirma se os valores ausentes ou nulos foram devidamente preenchidos com o padrão 'Sem Informação'.
2. **Tratamento e Padronização da Base Complementar (RES47_2021.xlsx):** Para integrar dados históricos específicos do relatório RES47, a planilha RES47_2021.xlsx foi lida ignorando linhas de títulos institucionais para reorientar os cabeçalhos. Em seguida, foram removidas colunas vazias sem rótulo ('Unnamed: 1' e 'Unnamed: 3'). Na coluna 'Unid. Medida', aplicou-se o dicionário de padronização siglar para converter termos minúsculos em caixa alta (como 'un' para 'UN', 'kg' para 'KG', 'l' para 'L'), preenchendo eventuais lacunas com 'Sem Informação' e exportando a base tratada como arquivo_novo_final.csv.

3. **Auditoria Temporal Final e Expurgos Residuais em Movimentações (RES82):** Na etapa final de auditoria temporal, o código converteu para o formato datetime a coluna 'Emissão' do relatório de movimentações (arquivo7) e inspecionou a lista de anos únicos presentes na base. Identificada a presença de registros residuais isolados pertencentes aos anos de 2011, 2020 e 2021, aplicou-se um filtro de exclusão para expurgar tais linhas, assegurando a estrita aderência ao recorte temporal do projeto. Por fim, as colunas de data 'Emissão' e 'Entrada' foram reformatadas para o padrão brasileiro ('%d/%m/%Y'), garantindo a perfeita sincronização temporal de todo o banco de dados.

Figura 36 - Carga Geral de Validação e Padronização da Base Complementar



```
import pandas as pd
from google.colab import drive
drive.mount('/content/drive')
o = '/content/drive/My Drive/API/Tratamento_3_Final/'
```

Mostrar saída oculta

```
arquivo1 = pd.read_csv(o + 'Arquivo1_Final.csv', low_memory=False, sep=';', encoding='UTF-8')
arquivo2 = pd.read_csv(o + 'Arquivo2_3_4_Final.csv', low_memory=False, sep=';', encoding='UTF-8')
arquivo3 = pd.read_csv(o + 'Arquivo5_Final.csv', low_memory=False, sep=';', encoding='UTF-8')
arquivo4 = pd.read_csv(o + 'Arquivo6_Final.csv', low_memory=False, sep=';', encoding='UTF-8')
arquivo5 = pd.read_csv(o + 'Arquivo7_Final.csv', low_memory=False, sep=';', encoding='UTF-8')
arquivo6 = pd.read_csv(o + 'Arquivo8_Final.csv', low_memory=False, sep=';', encoding='UTF-8')
arquivo7 = pd.read_csv(o + 'Arquivo9_Final.csv', low_memory=False, sep=';', encoding='UTF-8')
arquivo8 = pd.read_csv(o + 'Arquivo10_Final.csv', low_memory=False, sep=';', encoding='UTF-8')
```

```
oo = '/content/drive/My Drive/API/Tratamento_3_Final/NOVO/'
```

```
Arquivo1 = pd.read_excel(oo + 'RES47_2021.xlsx')
```

```
Arquivo1.to_csv(oo + 'arquivo_novo.csv', index=False, sep=';', encoding='utf-8-sig')
```

```
Arquivo_novo = pd.read_csv(oo + 'arquivo_novo.csv', low_memory=False, sep=';', encoding='UTF-8')
```

```
Arquivo_novo = pd.read_csv(
    oo + 'arquivo_novo.csv',
    low_memory=False,
    sep=';',
    encoding='UTF-8',
    header=1
)

Arquivo_novo.head(2)
```

Mostrar saída oculta

```
Arquivo_novo = Arquivo_novo.drop(columns=['Unnamed: 1', 'Unnamed: 3'])
```

```
Arquivo_novo.head(2)
```

Mostrar saída oculta

```
print(Arquivo_novo['Unid. Medida'].unique())
```

Mostrar saída oculta

Figura 37 - Carga Geral de Validação e Padronização da Base Complementar Parte 2



```python
padronizacao = {
    'un': 'UN',
    'kg': 'KG',
    'l': 'L',
    'cj': 'CJ',
    'm': 'M',
    'sc': 'SC',
    'm²': 'M²',
    'bo': 'BO',
    'par': 'PAR',
    'm': 'M',
    'cx': 'CX',
    'ce': 'CE',
    'bl': 'BL',
    'jg': 'JG',
    'gal': 'GAL',
    'mh': 'MH',
    'rl': 'RL',
    'fl': 'FL',
    'fc': 'FC',
    'rm': 'RM',
    'pct': 'PCT',
    't': 'T',
    'pç': 'PC',
    'dz': 'DZ',
    'br': 'BR',
    'la': 'LA',
    'bd': 'BD',
    'fr': 'FR',
    'bb': 'BB',
    'tb': 'TB',
    'cil': 'CIL',
    'AR': 'AR'
}

Arquivo_novo['Unid. Medida'] = Arquivo_novo['Unid. Medida'].replace(padronizacao)

Arquivo_novo['Unid. Medida'] = Arquivo_novo['Unid. Medida'].fillna('Sem Informação')

print(Arquivo_novo['Unid. Medida'].unique())
```

Mostrar saída oculta

```python
Arquivo_novo = Arquivo_novo.fillna('Sem Informação')

Arquivo_novo = Arquivo_novo.map(
    lambda x: 'Sem Informação' if isinstance(x, str) and x.strip() == '' else x
)

Arquivo_novo.to_csv(
    oo + 'arquivo_novo_final.csv',
    index=False,
    sep=';',
    encoding='utf-8-sig'
)

print('Arquivo salvo com sucesso!')
```

Mostrar saída oculta

```python
pd.set_option('display.max_columns', None)
arquivo7.head(1)
```

Mostrar saída oculta

Figura 38 - Auditoria de Anos em 'Emissão'/'Entrada' e Expulso de Anos Residuais (2011, 2020 e 2021)



```
# Converte a coluna para data
arquivo7['Emissão'] = pd.to_datetime(
    arquivo7['Emissão'],
    dayfirst=True,
    errors='coerce'
)

# Mostra os anos diferentes
anos = arquivo7['Emissão'].dt.year.dropna().unique()

print('Anos encontrados:')
print(sorted(anos))

print('\nQuantidade de anos diferentes:')
print(len(anos))
```

Mostrar saída oculta

```
# Remove as linhas referentes a 2011, 2020 e 2021
arquivo7 = arquivo7[
    ~arquivo7['Emissão'].astype(str).str.strip().isin(['2011', '2020', '2021'])
].reset_index(drop=True)
```

```
arquivo7['Emissão'] = pd.to_datetime(
    arquivo7['Emissão'],
    errors='coerce'
).dt.strftime('%d/%m/%Y')
```

```
arquivo7['Entrada'] = pd.to_datetime(
    arquivo7['Entrada'],
    errors='coerce'
).dt.strftime('%d/%m/%Y')
```

```
arquivo7['Entrada'] = pd.to_datetime(
    arquivo7['Entrada'],
    dayfirst=True,
    errors='coerce'
)

anos = arquivo7['Entrada'].dt.year.dropna().unique()

print('Anos encontrados:')
print(sorted(anos))

print('\nQuantidade de anos diferentes:')
print(len(anos))
```

Mostrar saída oculta

```
pd.set_option('display.max_columns', None)
arquivo7.head(1)
```

Mostrar saída oculta

```
arquivo7 = arquivo7[
    ~arquivo7['Entrada'].astype(str).str.strip().isin(['2021'])
].reset_index(drop=True)
```

```
pd.set_option('display.max_columns', None)
arquivo7.head(1)
```

Mostrar saída oculta

4.2. Modelagem Dimensional e Dashboards em Power BI

A etapa de Business Intelligence transformou a base de dados tratada do ERP ALVO em um modelo dimensional analítico no formato Esquema Estrela (Star Schema), integrando as tabelas fato de movimentações e saldos às dimensões de materiais, centros de custo e calendário. Essa arquitetura permitiu fundamentar a tomada de decisão gerencial por meio de fórmulas em linguagem DAX (Data Analysis Expressions) voltadas à velocidade de rotação, valoração financeira e acompanhamento temporal dos investimentos.

4.2.1. Módulo 1: Giro de Estoque e Cobertura

O objetivo deste módulo é calcular a velocidade com que os materiais circulam pelo almoxarifado e o tempo médio que permanecem armazenados, otimizando o espaço físico do galpão.

Arquitetura de Medidas DAX do Giro

- **Total de Consumo:** Isola e soma todas as quantidades de materiais que saíram efetivamente do estoque por meio de operações de consumo ou retirada no período selecionado.

```
Total Consumo =
CALCULATE(
    SUM(fMovimentacoes[Quantidade]),
    fMovimentacoes[Tipo_Movimento] = "Consumo"
)
```

- **Saldo Inicial e Saldo Final:** Calculam o volume acumulado de estoque no primeiro e no último dia do intervalo de datas filtrado, desconsiderando filtros visuais restritivos e baseando-se na linha temporal da tabela calendário.

```
Saldo Inicial =
VAR DataInicio = MIN(dCalendario[Date])
RETURN
CALCULATE(
    SUM(fMovimentacoes[Quantidade]),
    ALLSELECTED(dCalendario),
    dCalendario[Date] < DataInicio
)

Saldo Final =
VAR DataFim = MAX(dCalendario[Date])
RETURN
CALCULATE(
    SUM(fMovimentacoes[Quantidade]),
    ALLSELECTED(dCalendario),
    dCalendario[Date] <= DataFim
)
```

- **Estoque Médio:** Extrai a média aritmética entre o saldo inicial e o saldo final do período, servindo de denominador para o cálculo da velocidade de rotação.

```
Estoque Medio =
DIVIDE([Saldo Inicial] + [Saldo Final], 2)
```

- **Giro de Estoque:** Demonstra quantas vezes o item de estoque foi consumido e reposto no período analisado.

```
Giro de Estoque =
DIVIDE(
    [Total Consumo],
    [Estoque Medio],
    0
)
```

- **Tempo Médio de Permanência (Dias em Estoque):** Calcula em quantos dias o estoque atual se esgotará com base no ritmo de consumo vigente. Inclui tratamento de erro (BLANK) para evitar distorções estatísticas em itens parados.

```
Tempo Medio Permanencia (Dias) =
VAR DiasNoPeriodo = COUNTROWS(dCalendario)
RETURN
IF(
    ISBLANK([Giro de Estoque]) || [Giro de Estoque] <= 0,
    BLANK(),
    DIVIDE(DiasNoPeriodo, [Giro de Estoque])
)
```

4.2.1.1. Como Construir os Gráficos de Giro no Power BI

- **Cartões (KPI Cards):** Utilizados para exibir os valores consolidados de [Giro de Estoque] médio e [Tempo Medio Permanencia (Dias)] geral. Dão visibilidade imediata ao supervisor do almoxarifado.
- **Gráfico de Barras Agrupadas (Giro por Categoria/Material):** No Eixo Y, coloque a [Categoria] ou [Nome do Material]. No Eixo X, insira a medida [Giro de Estoque]. Isso permite identificar rapidamente quais materiais giram mais rápido e quais estão estagnados.
- **Tabela Detalhada por Item:** Coloque as colunas [ID_Material] e [Descricao], acompanhadas das medidas [Total Consumo], [Estoque Medio], [Giro de Estoque] e [Tempo Medio Permanencia (Dias)]. Aplique formatação condicional em escala de cores no tempo de permanência (destacando em vermelho os itens com maior tempo parados).

4.2.2. Módulo 2: Curva ABC por Consumo

O objetivo da Curva ABC é classificar automaticamente os materiais com base no impacto financeiro do consumo (Princípio de Pareto), separando-os em Classes A (80% do valor), B (15% do valor) e C (5% do valor) para direcionar negociações contratuais.

Arquitetura de Medidas DAX da Curva ABC

- **Valor Total do Consumo (Custo Total por Item):** Calcula o valor financeiro total consumido de cada material (Quantidade Consumida multiplicada pelo Custo Unitário).

```
Valor Consumo =
SUMX(
    fMovimentacoes,
    IF(
        fMovimentacoes[Tipo_Movimento] = "Consumo",
        fMovimentacoes[Quantidade] * fMovimentacoes[Custo_Unitario],
        0
    )
)
```

- **Percentual do Item sobre o Total:** Calcula a participação percentual do consumo daquele item em relação a todo o almoxarifado.

```
 % Consumo Individual =
DIVIDE(
    [Valor Consumo],
    CALCULATE([Valor Consumo], ALL(fMateriais))
)
```

- **Percentual Acumulado (Pareto):** Ordena os itens do maior para o menor consumo e acumula os percentuais progressivamente.

```
Percentual Acumulado =
VAR ValorAtual = [Valor Consumo]
VAR ValorAcima =
    CALCULATE(
        [Valor Consumo],
        FILTER(
            ALL(fMateriais),
            [Valor Consumo] >= ValorAtual
        )
    )
RETURN
DIVIDE(ValorAcima, CALCULATE([Valor Consumo], ALL(fMateriais)))
```

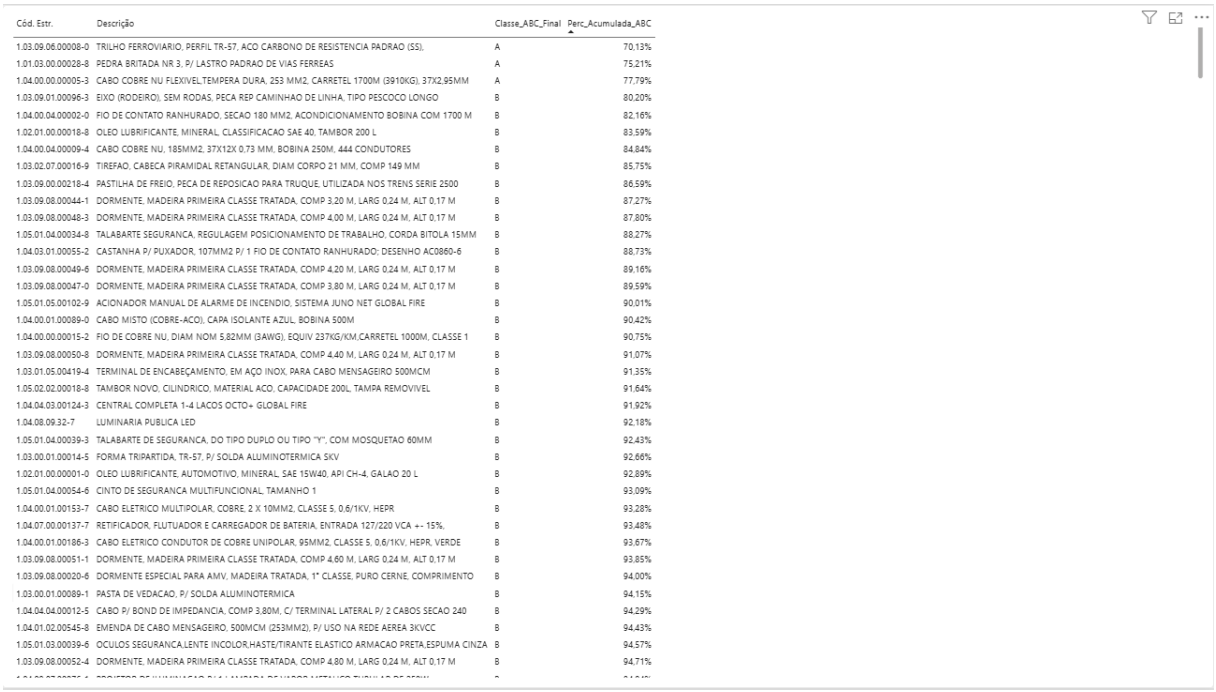
- **Classificação Automática (Curva ABC):** Atribui dinamicamente a letra A, B ou C com base no patamar do acumulado de Pareto.

```
Classe ABC =
VAR Acumulado = [Percentual Acumulado]
RETURN
SWITCH(
    TRUE(),
    Acumulado <= 0.80, "A",
    Acumulado <= 0.95, "B",
    "C"
)
```

4.2.2.1. Como Construir os Gráficos de Giro no Power BI

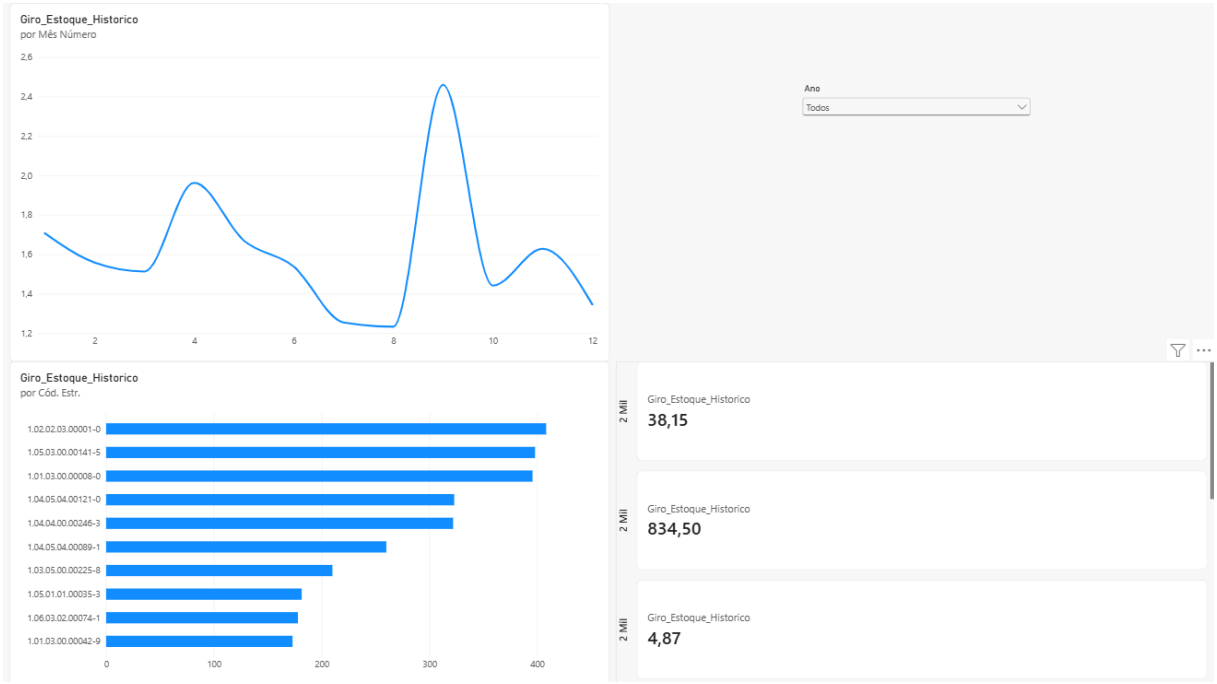
Matriz de Custos ABC: Crie uma matriz colocando nas linhas a hierarquia [Classe ABC] -> [Nome do Material], e nos valores: [Valor Consumo], [% Consumo Individual] e [Percentual Acumulado].

Figura 39 – PowerBI Página 1



Cód. Estr.	Descrição	Classe_ABC_Final	Perc_Acumulada_ABC
1.03.09.06.00008-0	TRILHO FERROVIARIO, PERFIL TR-57, ACO CARBONO DE RESISTENCIA PADRAO (SS),	A	70.13%
1.01.03.00.00028-8	PEDRA BRITADA NR 3, P/ LASTRO PADRAO DE VIAS FERREAS	A	75.21%
1.04.00.00.00005-3	CABO COBRE NU FLEXIVEL,TEMPERA DURA, 253 MM2, CARRETEL 1700M (3910KG), 37X2,95MM	A	77.79%
1.03.09.01.00096-3	EIXO (RODEIRO), SEM RODAS, PECA REP CAMINHAO DE LINHA, TIPO PESCOCO LONGO	B	80.20%
1.04.00.04.00002-0	FIO DE CONTATO RANHURADO, SECAO 180 MM2, ACONDICIONAMENTO BOBINA COM 1700 M	B	82.16%
1.02.01.00.00018-8	OLEO LUBRIFICANTE, MINERAL, CLASSIFICACAO SAE 40, TAMBOR 200 L	B	83.59%
1.04.00.04.00009-4	CABO COBRE NU, 185MM2, 37X12X 0,73 MM, BOBINA 250M, 444 CONDUTORES	B	84.84%
1.03.02.07.00016-9	TIREFAO, CABECA PIRAMIDAL RETANGULAR, DIAM CORPO 21 MM, COMP 149 MM	B	85.75%
1.03.09.00.00218-4	PASTILHA DE FREIO, PECA DE REPOSICAO PARA TRUQUE, UTILIZADA NOS TRENS SERIE 2500	B	86.59%
1.03.09.08.00044-1	DORMENTE, MADEIRA PRIMEIRA CLASSE TRATADA, COMP 3,20 M, LARG 0,24 M, ALT 0,17 M	B	87.27%
1.03.09.08.00048-3	DORMENTE, MADEIRA PRIMEIRA CLASSE TRATADA, COMP 4,00 M, LARG 0,24 M, ALT 0,17 M	B	87.80%
1.05.01.04.00034-8	TALABARTE SEGURANCA, REGULAGEM POSICIONAMENTO DE TRABALHO, CORDA BITOLA 15MM	B	88.27%
1.04.03.01.00055-2	CASTANHA P/ PUXADOR, 107MM2 P/ 1 FIO DE CONTATO RANHURADO; DESENHO AC0860-6	B	88.73%
1.03.09.08.00049-6	DORMENTE, MADEIRA PRIMEIRA CLASSE TRATADA, COMP 4,20 M, LARG 0,24 M, ALT 0,17 M	B	89.16%
1.03.09.08.00047-0	DORMENTE, MADEIRA PRIMEIRA CLASSE TRATADA, COMP 3,80 M, LARG 0,24 M, ALT 0,17 M	B	89.59%
1.05.01.05.00102-9	ACIONADOR MANUAL DE ALARME DE INCENDIO, SISTEMA JUNO NET GLOBAL FIRE	B	90.01%
1.04.00.01.00089-0	CABO MISTO (COBRE-ACO), CAPA ISOLANTE AZUL, BOBINA 500M	B	90.42%
1.04.00.00.00015-2	FIO DE COBRE NU, DIAM NOM 5,82MM (3AWG), EQUIV 237KG/KM,CARRETEL 1000M, CLASSE 1	B	90.75%
1.03.09.08.00050-8	DORMENTE, MADEIRA PRIMEIRA CLASSE TRATADA, COMP 4,40 M, LARG 0,24 M, ALT 0,17 M	B	91.07%
1.03.01.05.00419-4	TERMINAL DE ENCABEÇAMENTO, EM AÇO INOX, PARA CABO MENSAGEIRO 500MCM	B	91.35%
1.05.02.02.00018-8	TAMBOR NOVO, CILINDRICO, MATERIAL ACO, CAPACIDADE 200L, TAMPA REMOVIVEL	B	91.64%
1.04.04.03.00124-3	CENTRAL COMPLETA 1-4 LACOS OCTO+ GLOBAL FIRE	B	91.92%
1.04.08.09.32-7	LUMINARIA PUBLICA LED	B	92.18%
1.05.01.04.00039-3	TALABARTE DE SEGURANCA, DO TIPO DUPLO OU TIPO "Y", COM MOSQUETAO 60MM	B	92.43%
1.03.00.01.00014-5	FORMA TRIPARTIDA, TR-57, P/ SOLDA ALUMINOTERMICA 5KV	B	92.66%
1.02.01.00.00001-0	OLEO LUBRIFICANTE, AUTOMOTIVO, MINERAL, SAE 15W40, API CH-4, GALAO 20 L	B	92.89%
1.05.01.04.00054-6	CINTO DE SEGURANCA MULTIFUNCIONAL, TAMANHO 1	B	93.09%
1.04.00.01.00153-7	CABO ELETRICO MULTIPOLAR, COBRE, 2 X 10MM2, CLASSE 5, 0,6/1KV, HEPR	B	93.28%
1.04.07.00.00137-7	RETIFICADOR, FLUTUADOR E CARREGADOR DE BATERIA, ENTRADA 127/220 VCA +- 15%,	B	93.48%
1.04.00.01.00186-3	CABO ELETRICO CONDUTOR DE COBRE UNIPOLAR, 95MM2, CLASSE 5, 0,6/1KV, HEPR, VERDE	B	93.67%
1.03.09.08.00051-1	DORMENTE, MADEIRA PRIMEIRA CLASSE TRATADA, COMP 4,60 M, LARG 0,24 M, ALT 0,17 M	B	93.85%
1.03.09.08.00020-6	DORMENTE ESPECIAL PARA AMV, MADEIRA TRATADA, 1ª CLASSE, PURO CERNE, COMPRIMENTO	B	94.00%
1.03.00.01.00089-1	PASTA DE VEDACAO, P/ SOLDA ALUMINOTERMICA	B	94.15%
1.04.04.04.00012-5	CABO P/ BOND DE IMPEDANCIA, COMP 3,80M, C/ TERMINAL LATERAL P/ 2 CABOS SECAO 240	B	94.29%
1.04.01.02.00545-8	EMENDA DE CABO MENSAGEIRO, 500MCM (253MM2), P/ USO NA REDE AEREA 3KVCC	B	94.43%
1.05.01.03.00039-6	OCULOS SEGURANCA,LENTE INCOLOR,HASTE/TIRANTE ELASTICO ARMACAO PRETA,ESPUMA CINZA	B	94.57%
1.03.09.08.00052-4	DORMENTE, MADEIRA PRIMEIRA CLASSE TRATADA, COMP 4,80 M, LARG 0,24 M, ALT 0,17 M	B	94.71%

Figura 40 – PowerBI Página 2



4.3. Desenvolvimento e Implantação da Aplicação Web (CPTM Insight Flow App)

Para responder ao Item 7 da Sprint 2 do Backlog e permitir o acesso gerencial às análises sem a necessidade de licenças do Power BI, desenvolveu-se a aplicação CPTM Insight Flow App. A plataforma consome centraladamente os 8 arquivos CSVs higienizados na etapa de ETL em Python, garantindo unicidade das informações entre os almoxarifados. Para preservar a governança dos dados, a solução divide o acesso em dois perfis: Administrador, responsável pela carga e atualização das bases, e visitante, focado na navegação gerencial em modo leitura. A interface organiza a consulta em seis módulos lógicos (Status das Bases, Visão Geral do Estoque, Compras e Planejamento, Comparativo PCA x MOV, Fluxo de Entrega/Separação e Explorador com Curva ABC), permitindo que

supervisores e diretores monitorem os indicadores de rotatividade e valoração financeira a partir de qualquer dispositivo.

Figura 41 – Site Parte 1

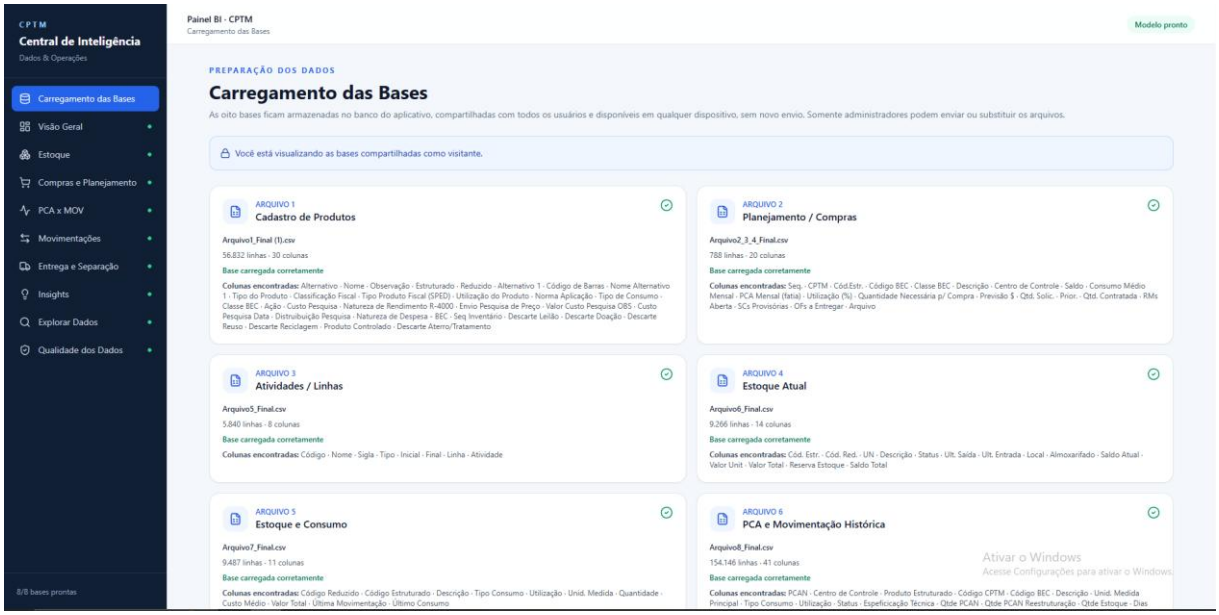


Figura 42 – Site Parte 2

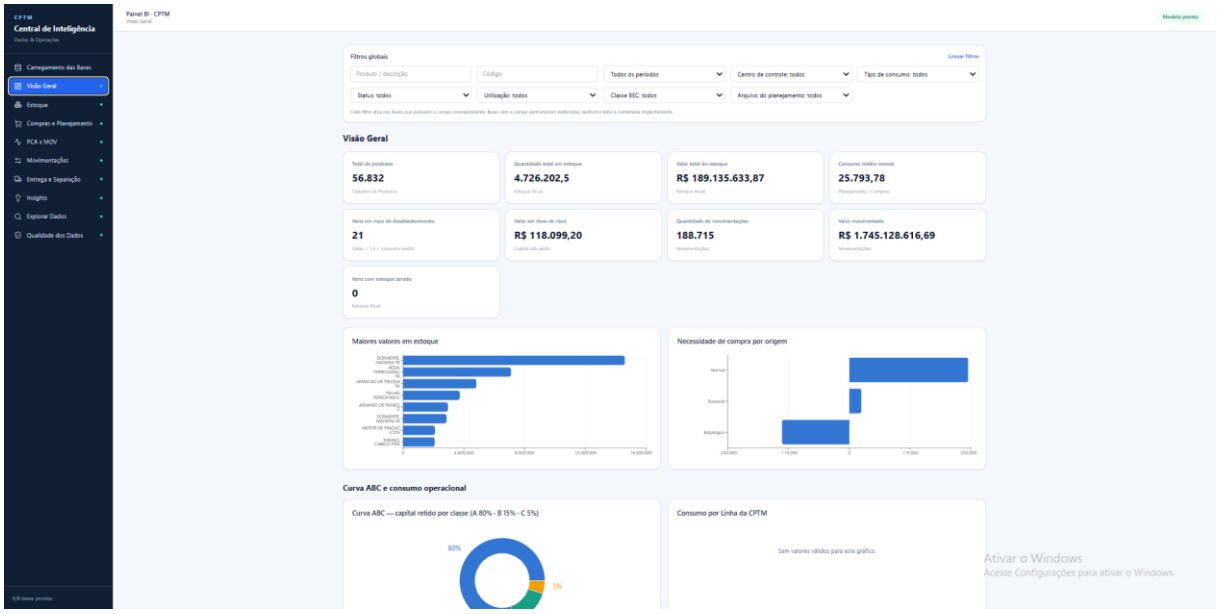


Figura 43 – Site Parte 3

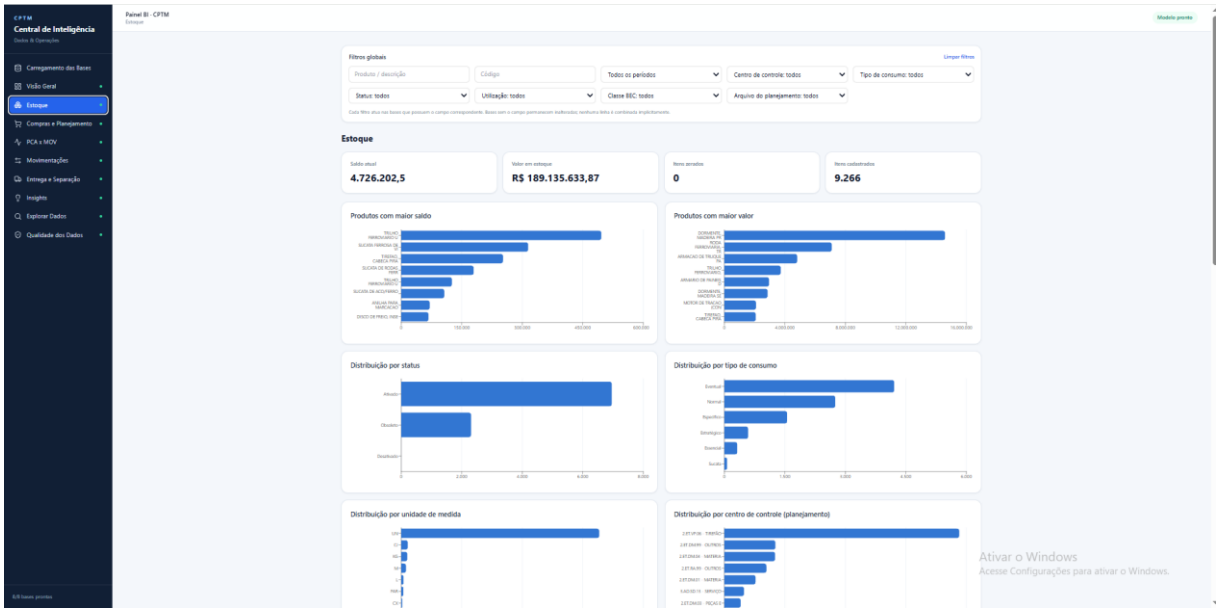


Figura 44 – Site Parte 4

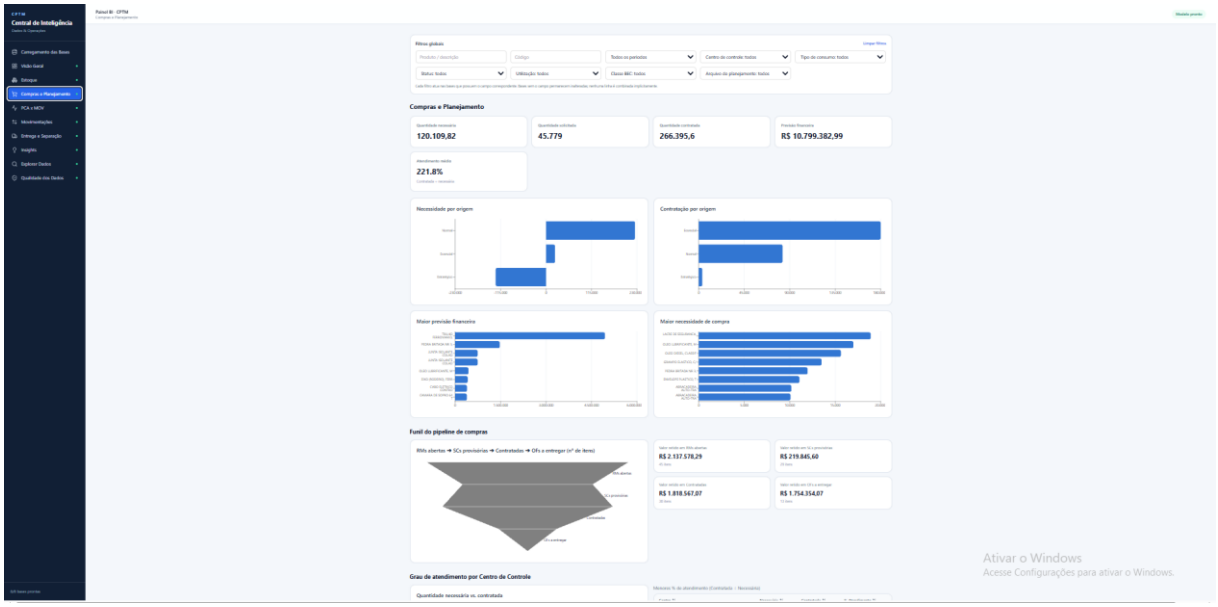


Figura 45 – Site Parte 5

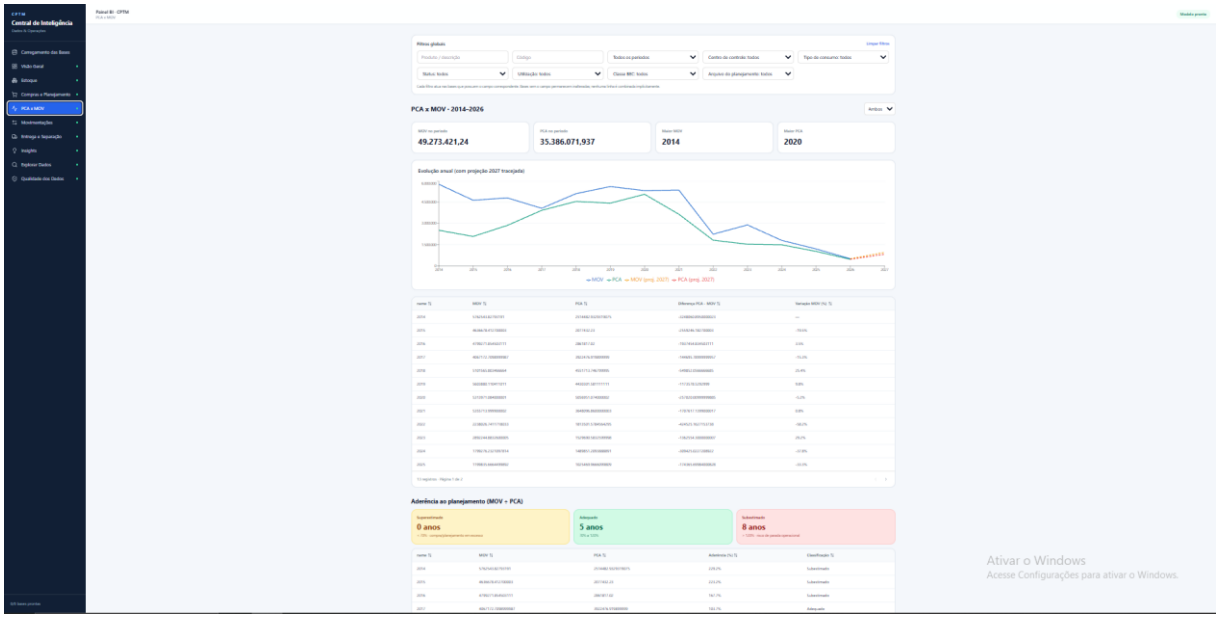


Figura 46 – Site Parte 6

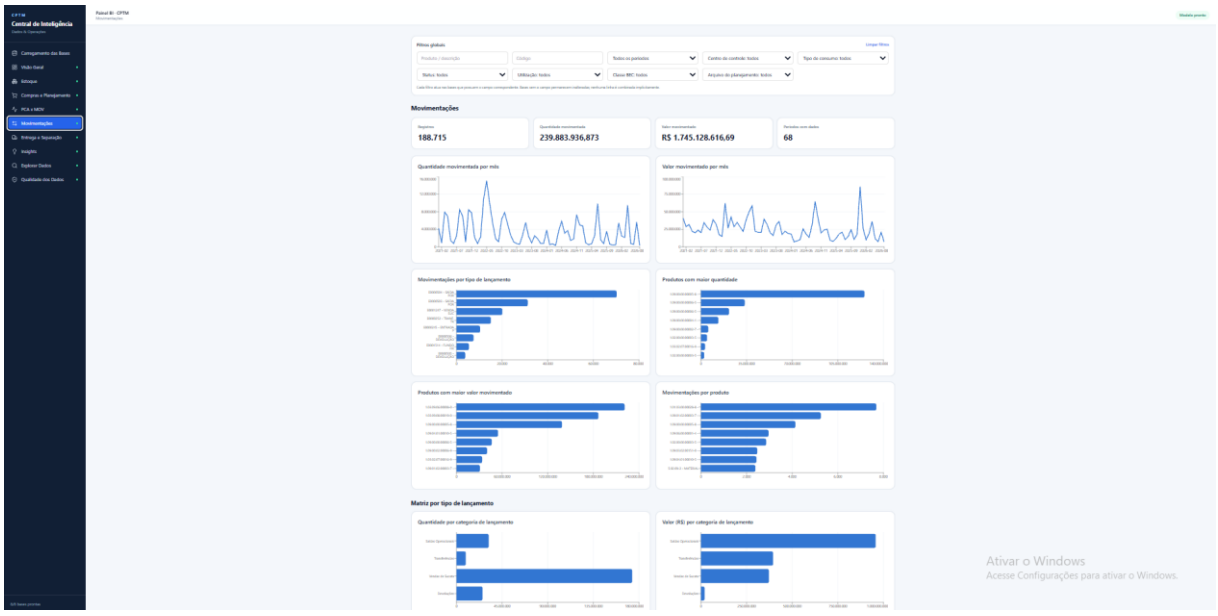


Figura 47 – Site Parte 7

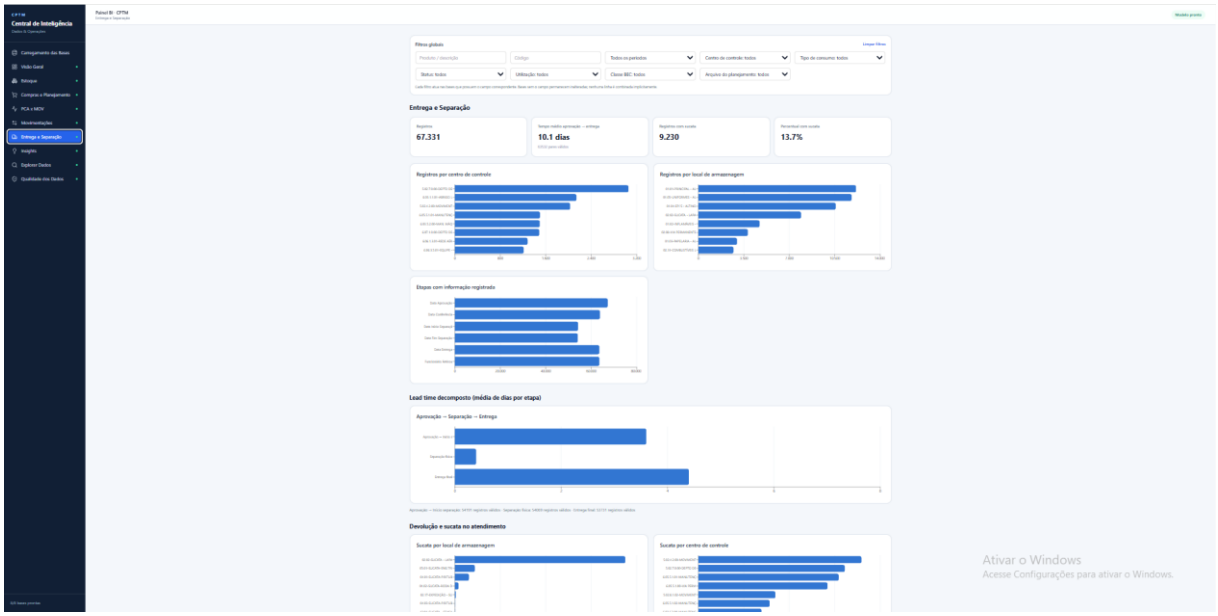


Figura 48 – Site Parte 8

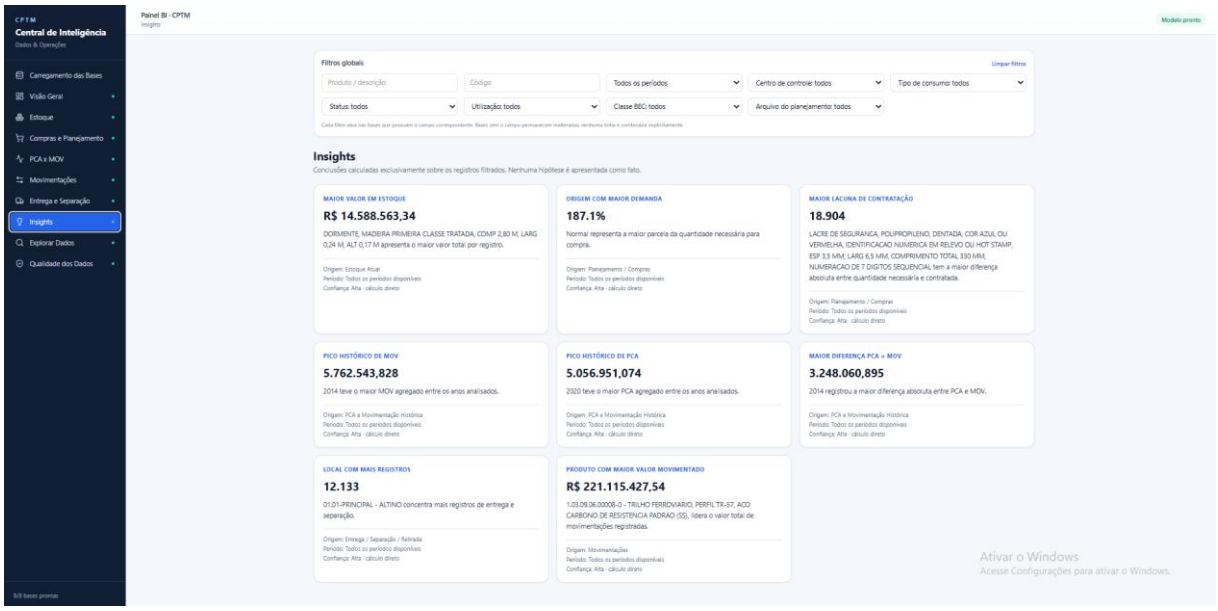


Figura 49 – Site Parte 9

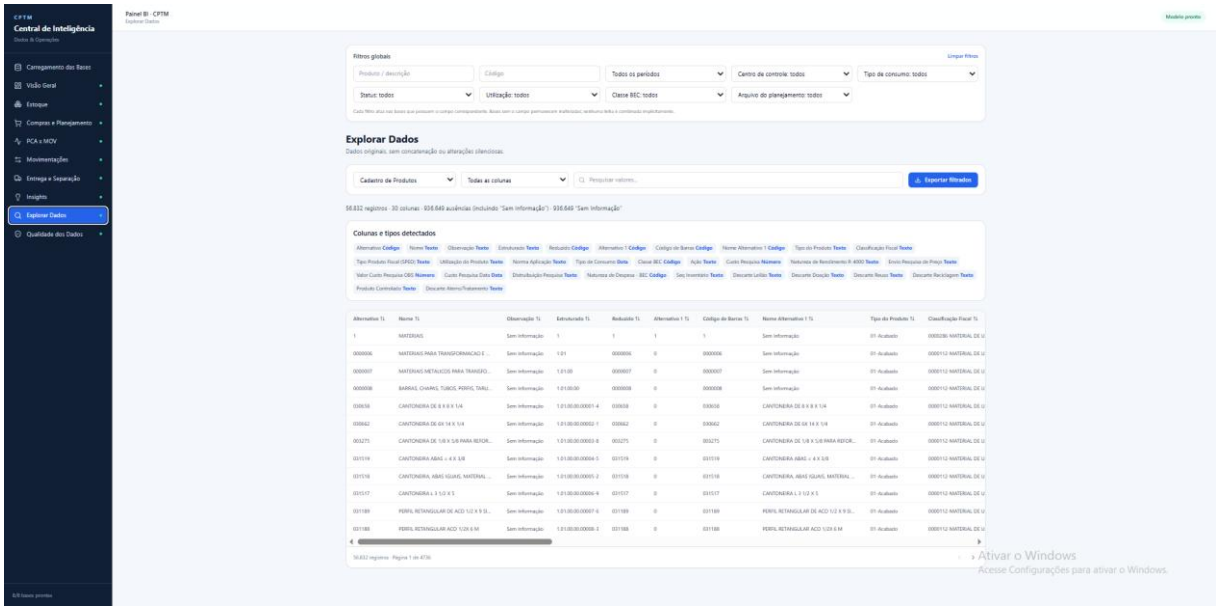
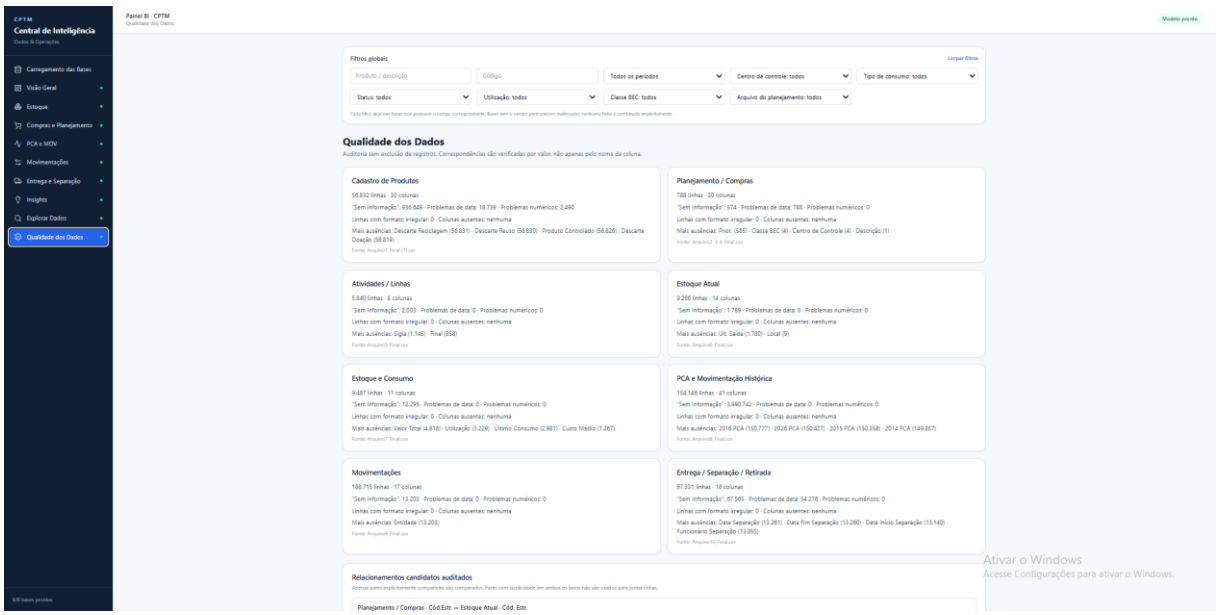


Figura 50 – Site Parte 10



5. Resultados esperados

A automação em Python dos 8 relatórios do ERP ALVO eliminou inconsistências de dados, alimentando os dashboards no Power BI e no site CPTM Insight Flow App. A solução entrega visibilidade em tempo real do estoque médio e do valor imobilizado, automação da Curva ABC para apoio a negociações contratuais e controle do giro de estoque para prevenir rupturas na manutenção da CPTM.

Referências

SÃO PAULO (Estado). Companhia Paulista de Trens Metropolitanos (CPTM). ERP ALVO: dados institucionais internos de controle de produtos em estoque. São Paulo, 2026.